

Graph-based Metric Learning per lo Scene Understanding

Deep Learning — Advanced Models and Methods

Track 26

Santi Lisi · Dario Lazzara · Francesco Granata

Università degli Studi di Catania

Dipartimento di Matematica e Informatica

Sommario

Il presente lavoro affronta il problema del recupero di scene visive per *similarità compositiva*: date due immagini, la nozione di somiglianza non è definita da texture o pixel, ma dalle entità presenti e dalle relazioni semantiche che le legano. Le scene del dataset GQA vengono rappresentate come grafi arricchiti tramite un'ontologia OWL gestita da un triple store Virtuoso, e proiettate in uno spazio metrico a 256 dimensioni mediante una *Graph Neural Network* (GINECONV) addestrata con funzione di perdita Triplet. La valutazione, condotta tramite un motore di retrieval FAISS su una ground truth semantica, dimostra la netta superiorità dell'approccio grafo-strutturato rispetto alle baseline visive: il modello migliore ottiene un'accuratezza (*Hit Rate*) dell'**81.58%** ad @20 sul subset (15k) e del **67.20%** sul fullset (55k), con un distacco di oltre **+37%** e **+26%** rispetto a CLIP. Infine, gli esperimenti di trasferimento *zero-shot* (81.71% ad @20) e i test di robustezza e spiegabilità evidenziano l'elevata capacità di generalizzazione e la coerenza dello spazio latente appreso.

Parole chiave — metric learning, graph neural network, scene graph, retrieval semantico, apprendimento contrastivo, semantic web, GQA

Indice

1	Introduzione	5
1.1	Motivazione	5
1.2	Obiettivo del Progetto	5
1.3	Contributi Principali	5
2	Dati, ontologia e ground truth semantica	6
2.1	Il dataset GQA e la pipeline dei dati	6
2.1.1	Caratteristiche del dataset GQA	6
2.1.2	Da triple RDF a oggetti di PyTorch Geometric	7
2.1.3	Il contratto dati verso i modelli	7
2.1.4	Suddivisione in split	7
2.2	Rappresentazione semantica: ontologia OWL e triple store	8
2.2.1	Modello ontologico e Protégé	8
2.2.2	Popolamento del triple store e arricchimento	8
2.2.3	Estensione al Full-Set per il Deep Learning	9
2.3	Ground truth semantica e selezione dei positivi	9
2.3.1	Rappresentazione della scena come oggetti e triple	9
2.3.2	Il punteggio di similarità basato su Jaccard pesata	10
2.3.3	Il veto semantico sugli embedding di scena	12
2.3.4	Selezione dei positivi tramite mining	12
2.3.5	Calibrazione delle soglie con un giudice esterno	13
2.3.6	Positivi e matrice di rilevanza	15
3	Metodologia: rappresentazione e apprendimento metrico	15
3.1	Baseline visive non strutturate	15
3.1.1	ResNet-50 e CLIP come estremi dello spettro visivo	15
3.1.2	Encoder congelati e dimensionalità nativa	17
3.1.3	Allineamento tra immagini e grafi	18
3.2	Encoder a grafo: architettura e varianti convolutive	18
3.2.1	Architettura in tre stadi	18
3.2.2	Le varianti Rel e NoRel dell'operatore di convoluzione	19
3.2.3	Gestione delle scene prive di archi	20
3.3	Apprendimento contrastivo e strategie di loss	20
3.3.1	Costruzione di positivi e negativi	21
3.3.2	Le due loss a confronto: Triplet Margin e NT-Xent	21
3.3.3	Ottimizzazione e diagnostica del collasso	22
4	Valutazione sperimentale e analisi dei risultati	24
4.1	Protocollo sperimentale e disegno di ablation	24
4.1.1	Le esecuzioni principali	24
4.1.2	Circolarità della ground truth e contromisure	25

4.2	Valutazione del retrieval sul sotto-insieme controllato (15k)	25
4.2.1	Infrastruttura di ricerca vettoriale: FAISS	25
4.2.2	Metriche di valutazione del Ranking	26
4.2.3	Analisi quantitativa dei modelli a grafo sul Subset (15k)	26
4.2.4	Confronto con le baseline visive sul Subset (15k)	27
4.3	Scala del dataset (55k) e test zero-shot incrociati	27
4.3.1	Effetto dell'ampliamento della galleria sul Full-Set (55k)	28
4.3.2	Valutazione Incrociata (Cross-Evaluation Zero-Shot)	28
4.4	Robustezza strutturale e interpretabilità	30
4.4.1	Test di robustezza a perturbazioni strutturali (Self-Retrieval)	30
4.4.2	Interpretabilità delle decisioni tramite GNNExplainer	31
5	Estensioni di sistema e deployment	32
5.1	Estrazione dinamica di scene graph tramite VLM	32
5.1.1	Il problema dell'allineamento a vocabolario aperto	33
5.1.2	Ricostruzione della rappresentazione	33
5.1.3	Prototipo funzionante e limiti architetturali	33
5.2	Infrastruttura distribuita e deployment	33
5.2.1	Architettura a Microservizi e Pipeline	34
5.2.2	Orchestrazione Locale e Cloud IaC	35
6	Discussione e conclusioni	35
6.1	Discussione critica dei risultati e minacce alla validità	36
6.1.1	Lettura critica dei risultati e confronto tra fattori	36
6.1.2	Perché la Triplet Margin Loss supera sistematicamente NT-Xent	37
6.1.3	Limiti metodologici e minacce alla validità	37
6.1.4	Analisi dei pattern di fallimento (Failure Modes)	38
6.2	Conclusioni e sviluppi futuri	38
6.2.1	Sintesi del lavoro	38
6.2.2	Copertura degli obiettivi	38
6.2.3	Sviluppi futuri	39
A	Iperparametri completi	39
B	Riproducibilità	40
B.1	Ambiente di esecuzione	41
B.2	Semi e determinismo	41
B.3	Comandi	41
C	Esempi qualitativi di retrieval	42
C.1	Esempio di Retrieval 1: Successo su scena complessa	42
C.2	Esempio di Retrieval 2: Astrazione semantica	42
C.3	Esempio di Retrieval 3: Focus sui dettagli	43

C.4	Analisi di un caso d'errore (Failure Case)	43
-----	--	----

1 Introduzione

Negli ultimi anni, i sistemi di *Content-Based Image Retrieval* (CBIR) hanno tratto enorme beneficio dai progressi nel deep learning visivo. Tuttavia, i modelli di rappresentazione globale basati su Convolutional Neural Networks (CNN) o Vision Transformers (ViT) tendono a comprimere l'intera scena in un singolo embedding vettoriale denso, privilegiando l'apparenza globale, le texture e la frequenza statistica degli oggetti. In tali rappresentazioni, la struttura relazionale sottostante viene spesso collassata: scene contenenti gli stessi elementi in configurazioni logiche radicalmente differenti (ad esempio, «uomo cavalca cavallo» rispetto a «uomo accarezza cavallo») risultano topologicamente indistinguibili nello spazio delle feature.

Per superare questo limite fondamentale, il presente progetto affronta il task di recupero per *similarità compositiva*, in cui la nozione di somiglianza non è legata all'apparenza dei pixel ma alle entità presenti e alle relazioni semantiche e spaziali che le connettono. Rappresentando ciascuna scena visiva come uno *Scene Graph*, il problema viene riformulato come l'apprendimento di uno spazio metrico invariante rispetto a variazioni di punto di vista, illuminazione o stile visivo, ma rigorosamente sensibile alla topologia semantica.

1.1 Motivazione

I metodi classici di image retrieval fondati sull'allineamento testo-immagine (come CLIP) manifestano un noto effetto di *bag-of-words*, faticando a distinguere correttamente il ruolo sintattico delle entità e la direzionalità dei predicati. Al contrario, uno scene graph codifica esplicitamente:

- Le **entità** (nodi) corredate dai loro attributi e categorie concettuali;
- Le **relazioni** (archi tipizzati) che definiscono le interazioni spaziali, d'azione e comparative tra gli oggetti.

L'integrazione di tecnologie del *Semantic Web* (ontologie OWL e triple store RDF) consente inoltre di disambiguare concetti polisemici e strutturare la gerarchia delle classi, fornendo alle reti neurali su grafo un substrato informativo di qualità superiore rispetto a mere etichette testuali piatte.

1.2 Obiettivo del Progetto

L'obiettivo primario è la progettazione, l'addestramento e la valutazione empirica di una pipeline end-to-end di *Scene-to-Scene Retrieval*: data una scena di query (espressa come grafo), il sistema deve recuperare in tempo reale le scene più simili all'interno di una galleria su larga scala, massimizzando la precisione compositiva.

1.3 Contributi Principali

Il lavoro apporta i seguenti contributi originali:

1. **Ground Truth e Metrica Compositiva Deterministica:** Definizione di un

criterio rule-based per quantificare la somiglianza semantica tra grafi, impiegato sia per il mining controllato delle coppie in addestramento sia come ground truth oggettiva di valutazione (Sez. 2.3).

2. **Arricchimento Semantico e Modellazione a Grafo:** Integrazione di un'ontologia OWL gestita tramite Virtuoso per l'arricchimento dei nodi e studio di architetture GNN (GINECONV, SAGECONV) capaci di operare sia con che senza attributi sugli archi (Sez. 3.2).
3. **Protocollo di Contrastive Metric Learning:** Benchmark sistematico di funzioni di perdita contrastive (NT-Xent vs. Triplet Loss) e studio di ablazione sull'impatto dei singoli componenti (Sez. 3.3, 4.1).
4. **Motore di Retrieval ad Alte Prestazioni:** Realizzazione di un'infrastruttura basata su database vettoriale FAISS per il matching scalabile su decine di migliaia di embedding (Sez. 4.2).
5. **Valutazione Multi-Scala e Generalizzazione Zero-Shot:** Validazione su due ordini di grandezza (Subset 15k e Fullset 55k) con analisi incrociata di transfer learning tra domini (Sez. 4.3).
6. **Analisi di Robustezza e Spiegabilità:** Valutazione della resilienza a perturbazioni strutturali (dropout casuale dei nodi) e analisi di salienza tramite *GNNE explainer* per interpretare le decisioni del modello (Sez. 4.4).

2 Dati, ontologia e ground truth semantica

Questa sezione descrive la filiera completa di preparazione dei dati: dall'estrazione e partizionamento del dataset visivo GQA, alla sua espansione ontologica mediante il triple store Virtuoso, fino alla formulazione matematica della *ground truth* semantica impiegata sia durante l'addestramento contrastivo sia per la valutazione del retrieval.

2.1 Il dataset GQA e la pipeline dei dati

2.1.1 Caratteristiche del dataset GQA

Il dataset GQA della Stanford University offre un'ampia collezione di scene visive annotate. Per il nostro task di retrieval basato su grafi, abbiamo focalizzato l'attenzione sugli *scene graph* associati a ciascuna immagine, ignorando le componenti testuali di question-answering. Ogni scene graph descrive in modo strutturato gli oggetti presenti (nodi), i loro attributi e le relazioni spaziali o semantiche tra di essi (archi). Inizialmente lo sviluppo della pipeline è avvenuto su un sottoinsieme bilanciato di 15.410 grafi, per poi essere scalato al *full-set* di oltre 55.000 immagini, garantendo così una maggiore generalizzazione dei modelli di deep learning addestrati.

2.1.2 Da triple RDF a oggetti di PyTorch Geometric

- Nodi: vettore di feature a 768 dimensioni ottenuto passando la stringa “classe + super-classi + attributi” all’embedder testuale NOMIC (`nomic-embed-text-v1.5`).
- Archi: `edge_index` $[2, |E|]$ per la topologia, `edge_attr` con l’embedding NOMIC della relazione.
- Grafi privi di archi (circa il 6% del totale) restano validi come *bag of objects*.

La conversione dai dati in formato RDF ai tensori di PyTorch Geometric ha richiesto l’interrogazione del triple store Virtuoso. Tramite script Python sono state eseguite query SPARQL per estrarre tutti i nodi e gli archi. Per gestire il dataset *full-set* ed evitare problemi di timeout del ragionatore durante l’inferenza delle macro-categorie, le query sono state ottimizzate e parzialmente pre-calcolate. Il formato finale prodotto è un file ‘.pt’ contenente una lista di oggetti `Data`, dove ogni grafo encapsula topologia e feature semantiche vettoriali pronte per la rete GCN.

2.1.3 Il contratto dati verso i modelli

La pipeline semantica e la parte di deep learning si incontrano in un solo punto, il file di grafi che l’encoder riceve in ingresso, ed è su quel punto che conviene dichiarare un contratto esplicito. Ogni scena è un oggetto `Data` di PyTorch Geometric con `x` di forma $[|V|, 768]$, dove ogni riga è l’embedding NOMIC della stringa che descrive un oggetto, `edge_index` di forma $[2, |E|]$ per la topologia, `edge_attr` di forma $[|E|, 768]$ con l’embedding della relazione, e `image_id`, che riconduce la scena alla propria fotografia (sezione 3.1.3).

Il contratto viene verificato prima di ogni esperimento anziché assunto, perché un disallineamento su questa interfaccia è silenzioso. Se `x` avesse la forma attesa ma valori privi di contenuto semantico, la rete si addestrerebbe ugualmente e la loss scenderebbe ugualmente, e si otterrebbero embedding perfettamente funzionanti di un’informazione che non c’è. Il controllo riguarda quindi le forme, la natura effettivamente semantica delle feature — che non si legge dalla forma e va accertata sul contenuto — e la presenza di `edge_attr` come tensore popolato anziché azzerato. Quando qualcosa non torna i vettori vengono rigenerati dalle stringhe di partenza, che ogni grafo conserva, e la versione verificata resta la sola fonte per tutti gli esperimenti.

L’ultimo dei tre controlli tocca da vicino l’esperimento centrale del lavoro. Un encoder edge-aware come GINECONV somma l’embedding dell’arco al messaggio che lo attraversa, per cui relazioni tutte uguali farebbero degenerare la variante Rel nella variante NoRel con qualche parametro in più, e il confronto tra le due (sezione 4.1.1) misurerebbe zero per costruzione anziché per assenza di segnale.

2.1.4 Suddivisione in split

Gli split non sono un dettaglio organizzativo, ma la condizione che rende confrontabili modelli diversi addestrati sulla medesima scala di dati. Sono generati con seed fisso pari a 42, così

che la differenza misurata tra due metodi dipenda esclusivamente dal metodo e non da quali scene siano capitate nel test set. Occorre precisare che l'operazione di rimescolamento su scale diverse (il sub-set da $\sim 15k$ e il full-set da $\sim 55k$) genera array differenti: le 1000 *queries* del full-set non coincidono con le 771 del sub-set. La confrontabilità assoluta è quindi garantita solo tra modelli testati all'interno della stessa scala dimensionale.

Tabella 1: Suddivisione del dataset (Full-set, oltre 55.000 grafi).

Split	Proporzione / Quantità
Train	$\sim 65\%$
Validation	$\sim 10\%$
Gallery (test)	Restante ($\sim 23\%$)
Queries (test)	Esattamente 1000

La suddivisione in split per il *full-set* mantiene rigorosamente il seed 42 per garantire la riproducibilità. Le proporzioni adottate assegnano il 65% dei grafi al training set e il 10% al validation set. Il restante 25% costituisce il test set, il quale subisce una netta separazione: vengono isolate esattamente 1000 immagini come *Queries*, mentre tutte le rimanenti compongono la *Gallery*.

Questa asimmetria è voluta: la gallery funge da spazio di ricerca (*search space*), e deve essere sufficientemente ampia per rendere il task di retrieval impegnativo e realistico. Testare su sole 1000 queries contro una gallery di oltre 13.000 elementi previene il rischio di retrieval banali.

Mediamente, ogni grafo del dataset è composto da un numero variabile di nodi e archi; tuttavia permane una quota del $\sim 6\%$ di grafi privi di archi che la pipeline tratta come semplici *bag of objects*. Il vincolo principale da preservare è la generazione degli split in modo deterministico. Questo criterio assicura che, seppur i dataset di test differiscano tra sub-set e full-set, la validità e la confrontabilità dei risultati all'interno della medesima dimensione (sezione 4.3) restino sempre verificabili.

2.2 Rappresentazione semantica: ontologia OWL e triple store

2.2.1 Modello ontologico e Protégé

Il punto di partenza è stato un sottoinsieme bilanciato del dataset GQA (circa 16.000 immagini). Per superare i limiti di un vocabolario piatto, è stata modellata un'ontologia tramite **Protégé**, definendo circa 300 classi base e raggruppandole in 20 *macro-categorie* gerarchiche (es. *Animale*, *Veicolo*, *Arredamento*). Questa TBox definisce regole logiche complesse come transitività, *property chains* e classi disgiunte.

2.2.2 Popolamento del triple store e arricchimento

Il grafo e l'ontologia sono stati caricati in un triple store **Virtuoso** containerizzato. Tramite query SPARQL, è stato verificato il corretto funzionamento del ragionatore semantico (RDFS/OWL), che inferisce automaticamente le super-classi di ogni nodo.

2.2.3 Estensione al Full-Set per il Deep Learning

Mentre la validazione manuale è avvenuta sul subset, l'estrazione delle macro-categorie è stata scalata all'intero dataset (*full-set*) tramite script Python. Il testo di ogni nodo passato al modello di embedding linguistico (NOMIC) viene arricchito concatenando la classe, i suoi attributi visivi e la *macro-categoria* inferita. Questo garantisce che la Graph Convolutional Network (GCN) riceva in input feature vettoriali semanticamente dense.

2.3 Ground truth semantica e selezione dei positivi

GQA annota oggetti, attributi e relazioni, ma non contiene alcuna etichetta di “scenario”. Il metric learning ne ha però bisogno in due punti distinti: in training, per stabilire quali coppie di scene la rete debba avvicinare; in valutazione, come verità di riferimento rispetto a cui misurare il ranking prodotto dal retrieval. Definire due criteri diversi per i due scopi renderebbe il risultato incomparabile con sé stesso, perciò è stato costruito un unico criterio, implementato in `scene_similarity.py` e usato in entrambi i punti.

Concretamente, ciò che va prodotto è per ogni scena una breve lista di altre scene dichiarate simili — i suoi *positivi* — e, per la valutazione, una matrice che assegni a ogni coppia query-gallery un grado di rilevanza. Due grandezze ricorrono nel seguito per descrivere il risultato. La *copertura* è la quota di scene che ricevono almeno un positivo, mentre il numero di candidati *a pari merito* in cima alla graduatoria misura quanto il punteggio riesca a ordinarli anziché dichiararli equivalenti.

Il criterio è deliberatamente *rule-based* e opera su insiemi e conteggi ricavati dai grafi, senza alcun modello generativo nel ciclo. La conseguenza più importante non è l'efficienza ma il determinismo — lo stesso corpus produce sempre la stessa ground truth, condizione necessaria perché le ablation delle sezioni successive misurino il modello e non la variabilità del criterio di valutazione. Il percorso completo, dalla scena alla lista dei positivi, è riassunto in figura 1, e le sottosezioni che seguono ne ripercorrono le tappe nell'ordine.

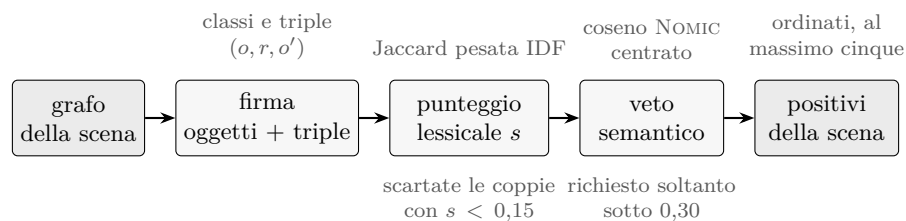


Figura 1: Le tappe che portano da una scena alla sua lista di positivi. La riga superiore indica che cosa ciascuno stadio calcola, quella inferiore che cosa scarta. Il punteggio non viene valutato su tutte le coppie possibili ma soltanto sui candidati che un indice invertito recupera a partire dagli oggetti più rari della firma, e lo stesso percorso, applicato tra query e gallery anziché all'interno del training set, produce la ground truth di valutazione.

2.3.1 Rappresentazione della scena come oggetti e triple

Ogni scena viene ridotta a una *firma* composta da due insiemi. Il primo raccoglie le classi degli oggetti presenti, estratte dalla prima porzione della descrizione testuale del nodo. Il

secondo raccoglie le *triple semantiche* ($classe_{src}, relazione, classe_{dst}$) costruite percorrendo gli archi.

Dalle triple sono escluse alcune relazioni puramente spaziali come *— to the left of* e simili. La motivazione è che descrivono l'inquadratura e non lo scenario, tanto che fotografare la stessa scena da due angolazioni ne stravolge le relazioni spaziali lasciando intatto il contenuto. Restano le relazioni che descrivono azione e funzione (*riding, holding, wearing*), cioè quelle che caratterizzano ciò che nella scena sta accadendo.

2.3.2 Il punteggio di similarità basato su Jaccard pesata

La similarità tra due scene A e B confronta separatamente le due metà della firma e ne combina i risultati:

$$s(A, B) = \alpha J_w(\mathcal{O}_A, \mathcal{O}_B) + (1 - \alpha) J_w(\mathcal{T}_A, \mathcal{T}_B), \quad \alpha = 0,5 \quad (1)$$

dove J_w è l'indice di Jaccard pesato, cioè il rapporto tra la somma dei pesi degli elementi in comune e la somma dei pesi dell'unione.

La ripartizione paritetica non è una convenzione, ma il risultato del confronto tra i criteri alternativi possibili, misurati sul validation set (tabella 2). Il solo insieme degli oggetti produce un punteggio denso e ordinabile ma cieco alla struttura, e non darebbe alla rete alcun incentivo a usare gli archi. Le sole triple risultano troppo sparse, coprono circa metà delle scene e lasciano centinaia di candidati a pari merito in cima alla graduatoria, perché con pochi elementi in comune il punteggio assume pochi valori distinti. Un raggruppamento in diciannove macro-classi è talmente generico che due scene qualsiasi si somigliano già per definizione, mentre il coseno tra le feature dei nodi copre tutto ma non correla con la struttura. La combinazione paritetica è l'unica configurazione al tempo stesso densa, ordinabile e correlata alla struttura del grafo.

Tabella 2: Confronto tra criteri di similarità, misurato sul validation set. *Sim. casuale* è il punteggio medio tra scene senza alcun legame (più basso è, più il criterio discrimina); *copertura* è la quota di scene con almeno un candidato; *pari merito* è il numero medio di candidati indistinguibili in cima alla graduatoria; *corr. struttura* è la correlazione del punteggio con la struttura del grafo.

Criterio	Sim. casuale	Copertura	Pari merito	Corr. struttura
Soli oggetti	0,045	92 %	1,3	0,30
Macro-classi (19)	0,385	86 %	5,5	0,04
Sole triple semantiche	0,001	55 %	694,3	1,00
Coseno delle feature	0,000	100 %	1,0	0,00
Misto (adottato)	0,023	92 %	1,2	0,30

Un caso limite riguarda i grafi poveri di archi, dove con una o due triple il denominatore della Jaccard strutturale è così piccolo che una coincidenza banale del tipo (*man, wearing, cap*) dominerebbe metà del punteggio. L'implementazione prevede per questa eventualità una soglia sul numero minimo di triple, rimasta però disattivata nelle esecuzioni definitive, dato

che il fenomeno è già contenuto a monte dall’attenuazione dei termini generici e dall’esclusione delle anchor più povere (sezione 2.3.4).

Resta da stabilire quanto valga ciascun elemento condiviso, dato che un elemento in comune non vale quanto un altro. Il peso di ciascun oggetto e di ciascuna tripla è la sua *inverse document frequency*, calcolata sull’intero corpus. Un *man* in comune, presente ovunque, sposta il punteggio di poco, mentre un *kite* in comune lo sposta molto.

L’IDF da sola però non distingue un elemento raro e significativo da uno raro e generico, dal momento che *clothes* o *guy* compaiono meno spesso di *man* senza per questo caratterizzare la scena. Serve quindi un secondo criterio, di natura lessicale anziché statistica, sullo stesso principio delle *stop-word* dell’information retrieval classico. Il criterio enumera quattro categorie chiuse del vocabolario di GQA — sfondi e *stuff* indifferenziato, capi di abbigliamento generici, termini che denotano la persona, parti del corpo — accomunate dal fatto di comparire in una scena a prescindere da che cosa la scena rappresenti. Sono quarantacinque termini in tutto, e il peso degli elementi che vi ricadono viene moltiplicato per un fattore di attenuazione pari a 0,15.

Due dettagli di questo meccanismo meritano attenzione. Il primo è che gli elementi attenuati non vengono *rimossi*. Eliminarli dal grafo impoverirebbe eccessivamente le scene con pochi oggetti, facendo perdere match altrimenti validi, mentre attenuarli lascia intatta la firma e ne riduce soltanto il contributo. Il secondo è che una tripla viene attenuata solo se *entrambi* i suoi estremi sono generici. In questo modo (*man, riding, horse*) conserva peso pieno, perché è informativa nel suo complesso nonostante il soggetto banale, mentre (*man, wearing, clothes*) viene correttamente svalutata.

Poiché le frequenze documentali erano già disponibili dal calcolo dell’IDF, si è valutata anche la via alternativa, ovvero derivare l’elenco dalla sola distribuzione delle etichette e attenuare gli oggetti al di sopra di una soglia di frequenza. Il confronto sugli stessi dati assegna alle due varianti la stessa precisione a meno del rumore sperimentale, ma alla seconda circa tre punti di copertura in meno, e la ragione di quella perdita chiarisce la differenza tra i criteri. La frequenza non separa un termine generico da uno che definisce lo scenario, e *grass, water, sand, snow* e *road* sono tra gli oggetti più frequenti del corpus proprio perché dicono “all’aperto”, “spiaggia” o “strada”. Attenuarli insieme a *clothes* e *guy* costa le coppie che quel solo elemento teneva insieme, mentre il criterio categoriale non li tocca, dal momento che nessuno dei cinque è uno sfondo indifferenziato né una parte del corpo.

Vale la pena osservare che, tra i due, è quello automatico il criterio dipendente dal corpus. Una soglia di frequenza si appoggia alla distribuzione delle etichette nello specifico insieme di scene e va ricalcolata quando l’insieme cambia, mentre l’appartenenza di *hand* alle parti del corpo è una proprietà del vocabolario e resta vera su qualunque sottoinsieme. Sul *quanto* attenuare la scelta si è invece rivelata poco influente, dato che facendo variare il fattore tra 0,05 e 1,0 la precisione resta a $60 \pm 1\%$ e la copertura si muove tra 78,8% e 82,2%. Il lavoro sostanziale lo fa dunque l’IDF, mentre l’attenuazione conta per *quali* elementi colpisce più che per l’intensità con cui li colpisce.

2.3.3 Il veto semantico sugli embedding di scena

La Jaccard, per quanto pesata, resta un criterio lessicale, poiché confronta le etichette per uguaglianza esatta. Due scene possono quindi condividere le parole giuste ed essere semanticamente scollegate, così come possono descrivere la stessa situazione con vocabolari diversi e non ottenere alcun punteggio. Serve un secondo giudizio, costruito su una strada indipendente.

Quel giudizio è l'embedding semantico della scena, ottenuto come media dei vettori NOMIC dei nodi ponderata con gli stessi pesi IDF. La ponderazione è necessaria per la stessa ragione vista sopra, dato che senza di essa il vettore finirebbe dominato da cielo, mani e persone, cioè proprio dagli elementi che non distinguono nulla.

Il passaggio decisivo, e meno ovvio, è la centratura. I vettori NOMIC grezzi occupano una regione ristretta dello spazio — il fenomeno noto come anisotropia — al punto che due scene prese a caso presentano coseno intorno a 0,91. Su una scala in cui tutto vale 0,9 nessuna soglia è informativa. Sottraendo la media del corpus prima di normalizzare, il coseno tra scene non correlate torna intorno a zero e la misura ridiventa utilizzabile.

Il veto interviene con un'asimmetria voluta, riassunta in figura 2. Se il punteggio lessicale è già forte ($s \geq 0,30$) la coppia viene accettata senza ulteriori verifiche, perché un'ampia sovrapposizione di oggetti e triple rare non ha bisogno di conferme, mentre se il punteggio è debole la coppia sopravvive solo quando il coseno semantico raggiunge almeno 0,55. Le coppie evidenti non vengono cioè toccate, e quelle borderline devono presentare due evidenze concordi, le parole e il significato.

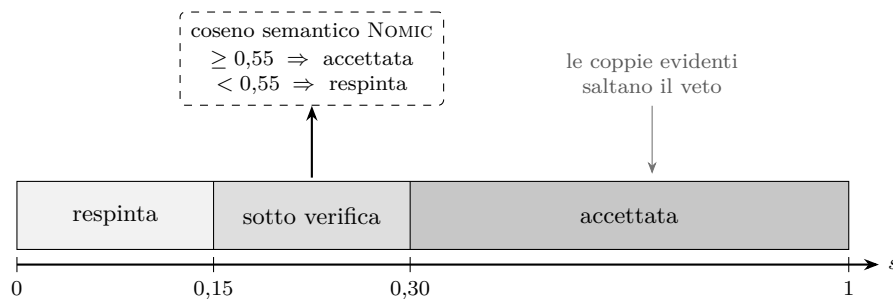


Figura 2: La regola asimmetrica del veto. Sotto la soglia minima la coppia non entra nemmeno in gioco, sopra la soglia di bypass il solo punteggio lessicale basta ad accettarla, e il secondo giudizio viene richiesto unicamente nella fascia intermedia, dove le parole in comune ci sono ma non bastano a decidere. È in quella fascia che si concentrano le coincidenze lessicali sospette, e la coppia sopravvive soltanto se le due evidenze concordano.

2.3.4 Selezione dei positivi tramite mining

Applicare il criterio a tutte le coppie possibili avrebbe costo quadratico nel numero di scene. Il mining procede quindi per candidati. Per ogni scena si selezionano i sei oggetti più rari della sua firma e, tramite un indice invertito costruito una volta sola, si recuperano soltanto le scene che ne condividono almeno uno, calcolando il punteggio completo su questo solo sottoinsieme. La scelta di partire dagli oggetti rari non è soltanto un'ottimizzazione, dal momento che si

Tabella 3: Parametri del criterio di similarità e criterio di scelta.

Parametro	Valore	Criterio di scelta
Peso oggetti / triple	0,5 / 0,5	confronto tra criteri (tabella 2)
Soglia minima s	0,15	gomito precisione/copertura
Positivi per anchor	max 5	varietà tra le epoche senza riempire di rumore
Attenuazione generici	0,15	copertura; precisione insensibile
Soglia del veto	0,55	otto configurazioni confrontate
Soglia di bypass	0,30	fissata insieme al veto; i match forti non richiedono conferma
Oggetti minimi	3	precisione 30 % contro 54-62 %
Oggetti rari per candidati	6	compromesso costo/copertura del mining

tratta degli stessi elementi che l'IDF premia, e il candidato scartato a monte è quindi anche quello che difficilmente avrebbe superato la soglia.

I candidati che superano la soglia minima e il veto vengono ordinati per punteggio decrescente e se ne conservano al massimo cinque. Il limite resta un massimo e non una quota da riempire, tanto che un'anchor con due sole scene realmente simili ne riceve due, perché completare la lista con candidati mediocri introdurrebbe rumore proprio dove il segnale è già scarso.

Le anchor con meno di tre oggetti vengono escluse dal mining. Una scena descritta da una o due classi generiche non possiede struttura sufficiente perché si possa affermare con onestà che un'altra scena rappresenti lo stesso scenario, dato che un grafo *flower + leaves* risulta compatibile tanto con un giardino quanto con la fantasia di un ombrellone. Non si tratta però di scartare dati, dato che quelle scene restano nel training, dove entrano come anchor con positivo ottenuto per augmentation e fungono da negativi per tutte le altre, e restano nella gallery di retrieval come ogni altra scena. L'unica cosa che non viene fatta è affermare una somiglianza che i dati non sostengono, perché un'etichetta sbagliata è un danno attivo mentre un'etichetta mancante è soltanto un po' meno segnale.

2.3.5 Calibrazione delle soglie con un giudice esterno

Lungo il percorso appena descritto compaiono gli otto parametri raccolti in tabella 3, e ciascuno è stato fissato su una misura anziché a sensazione. Il criterio non può però validare sé stesso, poiché qualunque soglia si scelga esso confermerà che le coppie selezionate sono simili, avendole selezionate con quel medesimo criterio. Serve un giudice esterno, ed è stato usato CLIP — che osserva i pixel e ignora completamente l'esistenza dei grafi, delle soglie e del veto.

Affinché il suo parere diventi un numero, si misura il coseno CLIP su circa ventimila coppie di immagini estratte a caso e se ne prende il novantacinquesimo percentile, che si colloca intorno a 0,64. La *precisione visiva* è allora la quota di coppie selezionate dal mining che supera quella soglia, e due immagini più simili del 95 % delle coppie casuali non lo sono per caso. Un criterio che scegliesse a caso otterrebbe per costruzione il 5 %, mentre la configurazione adottata raggiunge circa il 60 %, dodici volte tanto.

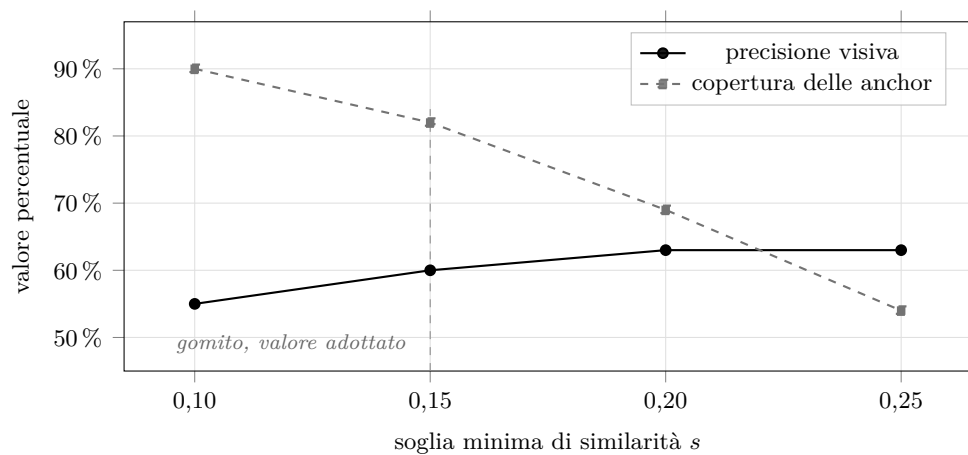


Figura 3: Le due grandezze in tensione al variare della soglia minima, con CLIP come giudice visivo esterno. La precisione visiva cresce fino a 0,20 e poi si arresta, mentre la copertura scende in modo pressoché lineare su tutto l'intervallo. A 0,10 entrano coppie mediocri e oltre 0,20 la precisione è saturata, per cui il valore adottato è il punto oltre il quale si continuerebbe a pagare copertura senza guadagnare precisione.

Precisione e copertura sono però in tensione tra loro, perché soglie severe producono coppie ottime per poche scene mentre soglie larghe coprono tutti lasciando entrare rumore. La regola di scelta adottata è sempre la stessa, cioè il *gomito* della curva (*elbow*) — il punto oltre il quale la precisione smette di crescere in modo apprezzabile mentre la copertura continua a scendere. L'ablation della soglia minima (figura 3) lo mostra con chiarezza, perché tra 0,20 e 0,25 la precisione è ferma e si perdono quindici punti di copertura, mentre tra 0,10 e 0,15 si guadagnano cinque punti di precisione perdendone otto di copertura. Oltre 0,20 le due curve si incrociano, e insistere significherebbe scambiare copertura contro nulla.

Lo stesso ragionamento è stato applicato al veto semantico, con otto configurazioni confrontate tra 0,0 e 0,65. Le sue due soglie sono però state fissate insieme, come un unico punto operativo, perché non sono parametri indipendenti — il veto agisce soltanto al di sotto della soglia di bypass, per cui spostare quest'ultima cambia *quante* coppie vengono sottoposte al secondo giudizio e non quanto bene quel giudizio discrimini. Il valore 0,30 non è quindi il risultato di un'ottimizzazione, ma la soglia oltre la quale una conferma semantica sarebbe ridondante.

La soglia sul numero minimo di oggetti è stata invece fissata stratificando la precisione per ricchezza dell'anchor, dove vale circa 30% al di sotto dei tre oggetti contro 54–62% al di sopra.

Resta un limite da dichiarare. CLIP giudica l'aspetto visivo e non la composizione, così che due scene con la stessa struttura logica ma pixel molto diversi — un kitesurfer e uno sciatore d'acqua — vengono conteggiate come errori del criterio pur essendo esattamente il tipo di somiglianza che l'encoder a grafo dovrebbe cogliere meglio dei pixel. Il 40% di coppie non confermate non è quindi tutto rumore, e il giudice va usato per confrontare configurazioni tra loro (“questa soglia è migliore di quella”) più che per attribuire un valore assoluto alla qualità del criterio.

2.3.6 Positivi e matrice di rilevanza

Lo stesso criterio, applicato tra l'insieme delle query e quello della gallery, produce la ground truth usata dal modulo di retrieval. Viene esposta in due forme affiancate, cioè una matrice di rilevanza continua, che conserva il punteggio e consente le metriche sensibili al ranking, e una matrice binaria ottenuta per soglia, sulla quale si calcolano precisione e richiamo ai primi k risultati. Entrambe sono calcolate una volta sola e distribuite come file insieme ai parametri usati per generarle, in modo che la valutazione non debba rieseguire il mining e che tutti i metodi confrontati siano giudicati esattamente sulla stessa verità di riferimento.

Un dettaglio dell'implementazione merita di essere dichiarato, perché ha conseguenze sui numeri. I pesi IDF e la centratura degli embedding di scena sono calcolati sull'unione di query e gallery, non separatamente sui due lati. Frequenze documentali stimate su due insiemi distinti assegnerebbero allo stesso oggetto due pesi diversi a seconda del lato in cui compare, e due medie di centratura diverse renderebbero i coseni delle due metà non confrontabili tra loro. Anche qui, come nel mining, la riduzione del costo passa da un indice invertito, dal momento che due scene senza alcun oggetto in comune hanno Jaccard nulla su entrambe le metà e non vale la pena valutarle.

Va infine dichiarato un limite che non riguarda la costruzione della ground truth ma l'uso che se ne fa. Un metro derivato dai grafi favorisce per costruzione i modelli che leggono i grafi, e il modo in cui il problema viene circoscritto è discusso in sezione 4.1.

Alcune coppie reali, riportate in figura 4, rendono concreto il funzionamento dei due giudizi combinati.

3 Metodologia: rappresentazione e apprendimento metrico

Questo capitolo formalizza la metodologia proposta per la mappatura delle scene visive in uno spazio metrico comune. Vengono anzitutto introdotte le baseline visive non strutturate (RESNET e CLIP), definite come termini di paragone per quantificare il valore aggiunto della struttura a grafo. Successivamente, viene presentata l'architettura dei Graph Encoder parametrici (SAGECONV e GINECONV) e le relative strategie di apprendimento contrastivo (NT-Xent e Triplet Margin).

3.1 Baseline visive non strutturate

Prima ancora di costruire l'encoder a grafo si è ritenuto necessario sapere quanto si ottenga dalla sola immagine, ignorando la struttura esplicita della scena. Una rete che lavora sui pixel non compete con la rappresentazione tramite grafo, ma ne fissa il termine di paragone, senza il quale l'affermazione che la struttura aggiunga qualcosa resterebbe indimostrata.

3.1.1 ResNet-50 e CLIP come estremi dello spettro visivo

Una baseline debole è facile da battere e rende il confronto poco credibile; una baseline unica, per quanto forte, non dice dove si collochi il risultato lungo la scala delle rappresentazioni visive



(a) $s = 0,83$, coseno $0,81$. In comune *surfboard*, *water*, *wetsuit* e le triple (*man*, *on*, *surfboard*) e (*man*, *wearing*, *wetsuit*).



(b) $s = 0,84$, coseno $0,74$. In comune *elephant*, *grass*, *trunk* e la tripla (*elephant*, *in*, *grass*).



(c) $s = 0,25$, coseno $-0,53$. L'unico oggetto non generico in comune è *cloud*.



(d) $s = 0,27$, coseno $-0,45$. L'unico oggetto non generico in comune è *park*, con due significati diversi.

Figura 4: Quattro coppie reali del corpus di valutazione. Nella riga superiore il punteggio lessicale supera la soglia di bypass e la coppia viene accettata senza ulteriori verifiche, e le due evidenze concordano comunque, dato che il coseno semantico è alto. Nella riga inferiore il punteggio è sufficiente a superare la soglia minima ma resta sotto quella di bypass, per cui interviene il veto, e il coseno negativo respinge la coppia. Sono esattamente i casi che la sola Jaccard non saprebbe distinguere da sé, dal momento che in tutti e quattro i casi le due scene condividono delle etichette.

possibili. Per questo sono state adottate due baseline scelte deliberatamente agli estremi.

ResNet-50 pre-addestrata su ImageNet rappresenta l’approccio classico basato sull’apparenza. La rete viene caricata con i pesi `IMAGENET1K_V2`, la head di classificazione viene sostituita da un’identità e si preleva l’uscita del *global average pooling*, un vettore a 2048 dimensioni. È stata addestrata a riconoscere un oggetto dominante per immagine, e le sue feature descrivono di conseguenza texture, forme e categoria principale, non le relazioni tra più oggetti. È la baseline che ci si aspetta di superare, e il margine con cui la si supera misura quanto la composizione conti al di là dell’apparenza.

CLIP nella variante `ViT-B/32` rappresenta invece la semantica olistica. L’embedding a 512 dimensioni si ottiene dal *vision transformer* seguito dalla proiezione multimodale, cioè dallo stesso spazio in cui il modello è stato allineato al testo su circa quattrocento milioni di coppie immagine-didascalia. Questa supervisione linguistica fa sì che CLIP colga concetti di scena — un mercato, una spiaggia, una partita — che una rete addestrata a classificare oggetti non esprime. È la baseline difficile, ed è anche il giudice esterno usato per calibrare la ground truth (sezione 2.3). I due ruoli restano distinti pur essendo affidati allo stesso modello, perché la calibrazione avviene sulle coppie selezionate dal mining e non sulle metriche di retrieval.

Entrambi gli encoder normalizzano l’uscita in norma ℓ_2 , così che l’indicizzazione avvenga nello stesso regime per tutti i metodi confrontati, e ciascuno conserva il proprio preprocessing ufficiale, quello distribuito con i pesi `IMAGENET1K_V2` per ResNet e il `CLIPImageProcessor` del checkpoint per CLIP. Imporre un ridimensionamento e una normalizzazione comuni avrebbe portato almeno uno dei due modelli a lavorare fuori dal regime statistico in cui è stato addestrato, penalizzandolo per una ragione estranea alla rappresentazione.

3.1.2 Encoder congelati e dimensionalità nativa

Le due baseline sono usate come estrattori di feature puri, senza alcun fine-tuning sui dati del progetto, e la scelta è deliberata. Sottoporre a fine-tuning ResNet o CLIP sulle immagini di GQA con una loss contrastiva insegnerebbe loro un’invarianza *visiva*, cioè a considerare vicine due foto che condividono illuminazione, inquadratura o dominante cromatica, mentre l’encoder a grafo viene addestrato su una nozione di vicinanza *semantica*. Il confronto dopo un fine-tuning metterebbe allora a paragone due modelli che hanno ottimizzato obiettivi diversi, e la differenza misurata non sarebbe più attribuibile alla rappresentazione. Con gli encoder congelati la domanda resta invece netta, e riguarda quanta informazione compositiva sia già presente nei soli pixel.

Per la stessa ragione ogni metodo conserva la propria dimensionalità nativa e viene indicizzato nel proprio spazio. Proiettare tutti i vettori a una dimensione comune avrebbe reso le tabelle più ordinate, ma una proiezione casuale degrada l’informazione tanto più quanto più alta è la dimensione di partenza, e si sarebbe così penalizzata proprio la baseline più ricca, ottenendo un confronto favorevole per costruzione. La differenza viene quindi dichiarata anziché nascosta, e non compromette il confronto poiché la metrica di ricerca — il coseno tra vettori normalizzati — è la stessa in tutti e tre gli spazi.

3.1.3 Allineamento tra immagini e grafi

Le due baseline lavorano sulle immagini mentre l'encoder a grafo lavora sulle annotazioni, e i due insiemi non sono garantiti coincidenti. L'allineamento avviene tramite l'identificatore dell'immagine, che compare sia nel grafo sia nel nome del file su disco. In fase di estrazione ogni scena il cui file risulti assente viene esclusa e conteggiata, e l'archivio prodotto conserva accanto agli embedding la lista degli identificatori nello stesso ordine, così che il retrieval accoppi query e gallery per identità della scena e non per posizione nella matrice. Il conteggio non è una formalità, dal momento che una fotografia mancante ridurrebbe l'insieme su cui la baseline viene valutata e un confronto ignaro del fatto misurerebbe due metodi su due gallery diverse. Nel corpus usato per gli esperimenti (*full-set*), il numero di embedding visivi prodotti coincide esattamente con quello dei grafi presenti nello split. Nessuna scena è andata perduta a causa di file di immagine mancanti: i tre metodi, di conseguenza, sono stati valutati e confrontati in modo rigoroso esattamente sulle stesse scene, assicurando che nessuna variazione di performance sia imputabile a disallineamenti nelle *gallery*.

3.2 Encoder a grafo: architettura e varianti convolutive

L'encoder ha il compito di trasformare un grafo di dimensione variabile — le scene di GQA vanno da pochi nodi a diverse decine — in un vettore a dimensione fissa che rappresenti l'intera scena. La scelta progettuale decisiva non riguarda però la profondità o la larghezza della rete, ma se il message passing possa o meno *leggere le relazioni*, ed è da qui che nascono le due varianti su cui si regge l'ablation centrale del lavoro.

3.2.1 Architettura in tre stadi

Il modello è implementato in `gcn_encoder.py` come un unico modulo parametrico e si articola in tre stadi, riassunti in figura 5.

Il primo è il **message passing**, dove tre layer di convoluzione su grafo portano le feature dei nodi da 768 a 256 dimensioni, mantenendo poi tale larghezza. Ogni layer è seguito da batch normalization, attivazione ReLU e dropout con probabilità 0,3. Tre layer corrispondono a un raggio di ricezione di tre archi, sufficiente perché un nodo integri il proprio contesto rilevante, dato che nelle scene di GQA un oggetto e ciò con cui interagisce distano tipicamente uno o due passi, e aumentare la profondità espone al fenomeno dell'*over-smoothing*, per cui le rappresentazioni dei nodi convergono verso un valore comune e il grafo perde capacità discriminativa. La batch normalization, in particolare, non è un accessorio, perché i grafi hanno un numero di nodi molto variabile e senza di essa la scala delle attivazioni dipenderebbe dalla dimensione della scena anziché dal suo contenuto.

Il secondo stadio è il **pooling globale**, che riduce le rappresentazioni dei nodi a un solo vettore tramite media. La media è invariante rispetto all'ordine dei nodi — proprietà necessaria, perché in un grafo l'ordine non ha significato — ed è insensibile al numero di nodi, mentre con la somma una scena affollata produrrebbe un vettore di norma sistematicamente maggiore e la similarità rifletterebbe la densità dell'annotazione invece del contenuto. Il massimo, all'estre-

mo opposto, conserverebbe soltanto l'oggetto più attivo per ciascuna dimensione, scartando proprio l'informazione compositiva che il progetto intende misurare. Tutte e tre le opzioni restano disponibili come parametro, ma il confronto tra architetture avviene a pooling fissato sulla media.

Il terzo stadio è la **projection head**, un MLP a due layer con attivazione intermedia, seguita da normalizzazione ℓ_2 dell'uscita a 256 dimensioni. La normalizzazione fa sì che il prodotto scalare tra due embedding coincida con la loro similarità coseno, cioè che la quantità ottimizzata dalle loss sia la stessa calcolata in fase di ricerca (sezione 4.2.1).

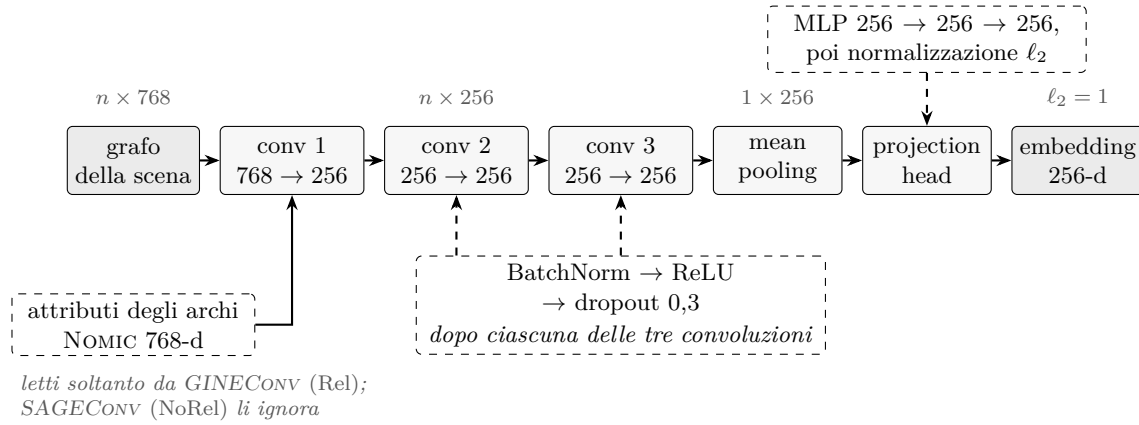


Figura 5: I tre stadi dell'encoder. Le convoluzioni portano le feature dei nodi a 256 dimensioni e le mantengono, il pooling riduce la scena a un solo vettore indipendentemente dal numero di nodi e la projection head produce l'embedding normalizzato usato nella ricerca. Il secondo ingresso in basso a sinistra è l'unica differenza tra le due varianti confrontate, dal momento che soltanto GINEConv legge l'embedding della relazione durante il message passing.

3.2.2 Le varianti Rel e NoRel dell'operatore di convoluzione

L'operatore di convoluzione è parametrico e definisce le due varianti confrontate nel lavoro.

SAGEConv, la variante *NoRel*, aggrega il vicinato osservando soltanto *che* due nodi sono collegati. Le relazioni contribuiscono alla rappresentazione unicamente attraverso la topologia che inducono, e la rete sa che l'uomo e il cavallo sono connessi ma non in che modo lo siano. È, di fatto, un *bag of objects* arricchito dalla struttura di adiacenza.

GINEConv, la variante *Rel*, è edge-aware. Nel message passing ogni vicino trasporta l'embedding NOMIC della relazione che lo collega, sommato alla propria rappresentazione prima di attraversare un MLP appreso a due layer, uno per ciascuna convoluzione. Le scene *uomo-cavalca-cavallo* e *uomo-accarezza-cavallo*, indistinguibili per la variante precedente, generano così messaggi diversi e possono ricevere embedding diversi (figura 6). Poiché le relazioni sono rappresentate dai vettori dello stesso embedder usato per i nodi, relazioni sinonime producono messaggi vicini senza che nessuno debba enumerarle in un vocabolario chiuso.

Le due varianti condividono tutto il resto, cioè stessa profondità, stesse dimensioni, stesso pooling, stessa head e stessi iperparametri di training. Nel codice si passa dall'una all'altra con un solo argomento sulla riga di comando, e la differenza di prestazione nel retrieval è

quindi attribuibile alla sola informazione relazionale. È questa la struttura dell’ablation *Rel contro NoRel* discussa in sezione 4.1.

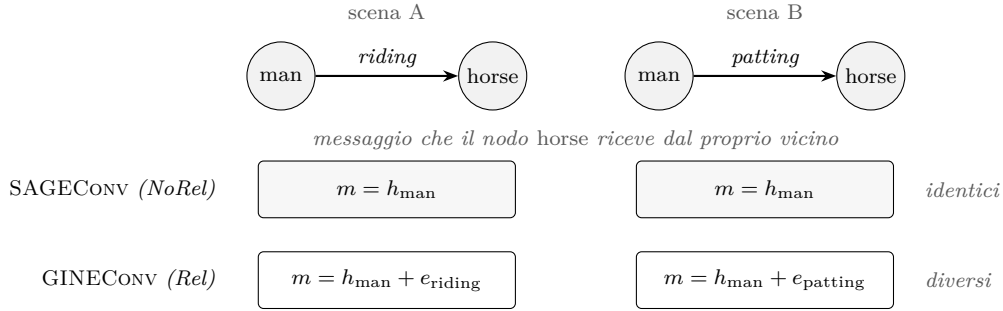


Figura 6: Il messaggio che il nodo *horse* riceve dal proprio vicino nelle due varianti. Le due scene hanno la stessa topologia e differiscono soltanto per l’etichetta dell’arco, per cui SAGECONV produce per entrambe lo stesso messaggio e non può che assegnare loro rappresentazioni vicine, mentre GINECONV somma al messaggio l’embedding della relazione e le mantiene distinguibili. È la differenza che l’ablation *Rel contro NoRel* intende misurare.

3.2.3 Gestione delle scene prive di archi

Circa il 6 % delle scene del corpus non possiede alcun arco, ed è una quota troppo alta perché la si possa ignorare. Una scena isolata di questo tipo non produce alcun messaggio, per cui ciascun nodo viene aggiornato soltanto a partire dalle proprie feature e il pooling ne restituisce la media. L’encoder la tratta cioè come un insieme di oggetti, che è esattamente quello che è, e la fa attraversare la rete insieme a tutte le altre senza alcun trattamento speciale.

Un secondo caso, distinto dal primo, riguarda i grafi che possiedono archi ma non l’attributo che li descrive. Un encoder edge-aware richiede formalmente un vettore per ogni arco e la sua assenza produrrebbe un errore di esecuzione, per cui la variante GINECONV sostituisce l’attributo mancante con un vettore nullo, cioè con una relazione “muta” che lascia il messaggio pari alla sola rappresentazione del vicino. Il caso non si presenta sui grafi annotati a mano, dove l’embedding della relazione viene generato insieme all’arco, ma è la condizione che rende il modello utilizzabile su grafi prodotti automaticamente (sezione 5.1), dove la copertura relazionale è per natura meno affidabile. L’effetto complessivo è un degrado continuo anziché un fallimento, dato che una scena parzialmente annotata usa le relazioni disponibili e ignora quelle assenti, senza che nulla nel codice debba distinguere i due regimi.

3.3 Apprendimento contrastivo e strategie di loss

L’obiettivo del training non è classificare ma organizzare uno spazio, portando vicine le scene che il criterio di sezione 2.3 riconosce come simili e lasciando lontane tutte le altre. Nessuna etichetta viene predetta, e il segnale di apprendimento nasce interamente dai confronti all’interno del batch.

3.3.1 Costruzione di positivi e negativi

Ogni scena del batch funge da anchor. Il suo positivo è una scena *diversa* scelta a caso tra i cinque candidati che il mining le ha associato, e il campionamento viene ripetuto a ogni epoca, così che nel corso del training la stessa anchor venga avvicinata a tutti i suoi candidati, uno per volta. La scelta è deliberata — fissare il positivo significherebbe insegnare alla rete una singola coppia, mentre alternarli la costringe a individuare ciò che quelle scene hanno in comune, cioè il *tipo* di scenario.

Quando un'anchor non possiede positivi affidabili — scene troppo povere di oggetti, oppure candidati tutti respinti dal veto — si ricade su una vista dell'anchor stessa ottenuta per augmentation, scartando il dieci per cento dei nodi e il venti per cento degli archi e azzerando un decimo delle componenti delle feature, con la sola cautela che una scena non scenda mai sotto i due nodi. Il fallback mantiene così la scena nel training anziché escluderla e le assegna un obiettivo onesto, l'invarianza a perturbazioni della propria struttura, senza affermare somiglianze che il criterio non sostiene, e la stessa augmentation costituisce da sola la modalità puramente self-supervised che il codice prevede come alternativa al mining. Non è comunque il caso prevalente, dato che nelle esecuzioni descritte poco meno del 90% delle scene ha ricevuto positivi dal mining, con una media di 4,6 candidati ciascuna.

I negativi non vengono selezionati esplicitamente, ma sono gli altri elementi della stessa matrice di similarità calcolata sul batch. Con batch da 128 elementi ogni anchor si confronta a ogni passo con oltre un centinaio di scene non correlate, e nel caso di NT-Xent, dove le due viste vengono concatenate, le righe della matrice diventano 256. È un numero sufficiente perché il segnale negativo sia informativo senza dover mantenere strutture dedicate come una memoria o una coda di negativi.

3.3.2 Le due loss a confronto: Triplet Margin e NT-Xent

Poiché la stessa idea di avvicinamento può essere formalizzata in modi profondamente diversi, si è scelto di implementarne due e di metterle a confronto come una delle ablation del lavoro.

Triplet Margin impone un vincolo locale, per cui l'anchor deve essere più vicina al proprio positivo che a un negativo, di almeno un margine fissato a 0,3. Le distanze sono calcolate come coseni tra embedding già normalizzati, così che la matrice dei prodotti scalari tra le anchor e i rispettivi positivi contenga sulla diagonale le coppie corrette e altrove tutti i negativi disponibili. Il negativo non viene scelto a caso ma tramite *semi-hard mining* (figura 7), ovvero si seleziona il più vicino tra i negativi che si trovano già dalla parte corretta. La scelta esclude deliberatamente il negativo più difficile in assoluto, che pure sarebbe disponibile, perché produce gradienti molto grandi generati spesso da coppie degeneri — scene in realtà simili, etichettate negative solo perché il mining non le ha accoppiate — e spinge la rappresentazione verso il collasso. Laddove nessun negativo semi-hard esista nel batch si ricade sul più difficile, per non lasciare l'anchor priva di segnale.

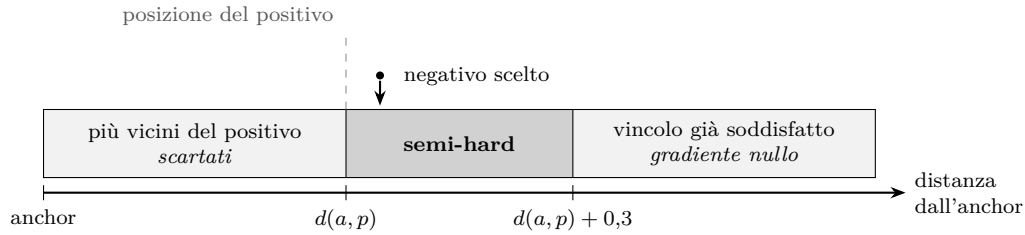


Figura 7: Da quale fascia proviene il negativo di ogni tripla. I negativi più vicini all’anchor del positivo stesso sono i più informativi ma anche quelli che con maggiore probabilità sono scene simili sfuggite al mining, e vengono esclusi. Oltre il margine il vincolo è già rispettato e la tripla non produce gradiente. Si sceglie quindi il negativo più vicino tra quelli che stanno già dalla parte corretta, e soltanto se la fascia centrale è vuota si ricade su uno della fascia di sinistra.

NT-Xent formula invece un obiettivo globale. Le due viste dello stesso elemento vengono concatenate e la matrice di tutte le similarità diventa il logit di una classificazione in cui la classe corretta è il positivo e tutte le altre posizioni del batch sono classi sbagliate. Ogni anchor è quindi confrontata simultaneamente con l’intero batch, non con un singolo negativo scelto.

La temperatura merita una motivazione esplicita, perché il valore adottato si discosta da quello convenzionale. Il valore 0,5 diffuso in letteratura è tarato sull’impostazione self-supervised classica, dove entrambe le viste sono versioni alterate della stessa immagine e per le quali è corretto pretendere una similarità molto alta. Qui le due viste sono invece la scena originale, che entra nella rete senza perturbazioni, e un positivo semantico recuperato dal mining, cioè una scena diversa e affine per contenuto con una quota misurata di rumore, che una temperatura alta tratterebbe con una severità che i dati non giustificano. Il valore 0,2 si colloca nell’intervallo 0,07–0,2 adottato in letteratura per il contrastive supervisionato, ed è stato scelto all’estremo prudente di tale intervallo.

Va infine sottolineato che i valori numerici delle due loss non sono confrontabili tra loro, essendo definite su scale diverse. Stabilire quale organizzzi meglio lo spazio è possibile solo attraverso le metriche di retrieval, ed è la ragione per cui il confronto tra loss compare tra le ablation e non tra le curve di training.

3.3.3 Ottimizzazione e diagnostica del collasso

L’ottimizzatore è AdamW con learning rate iniziale 10^{-3} e weight decay 10^{-4} , accompagnato da uno scheduler a coseno che porta il learning rate verso zero lungo l’intero training.

Lo scheduler è stato introdotto dopo una prima esecuzione da cento epoche a learning rate costante, nella quale la loss risultava ancora in lieve discesa al termine, avendo perso altri due centesimi nelle ultime venti epoche. Un simile andamento indica non convergenza e avrebbe reso qualsiasi confronto tra configurazioni una fotografia scattata a metà corsa. Portando il training a trecento epoche con learning rate decrescente, la fase finale avviene con aggiornamenti sempre più piccoli e la loss raggiunge un plateau osservabile, tanto che nelle ultime venti epoche si muove ormai soltanto sulla terza cifra decimale (figura 8). Le configurazioni

confrontate nelle ablation sono quindi tutte realmente arrivate a convergenza, e la differenza tra loro non dipende da dove si sia deciso di fermarsi.

Il rischio caratteristico dell'apprendimento contrastivo è il collasso della rappresentazione, che si verifica quando la rete scopre che assegnare a tutte le scene lo stesso vettore soddisfa banalmente il vincolo sui positivi. La loss scende allora mentre l'embedding perde ogni capacità discriminativa. Poiché il fenomeno non è visibile dalla sola curva della loss, alla fine di ogni epoca vengono registrate due grandezze diagnostiche. La prima è il coseno medio fuori diagonale, cioè tra elementi *diversi* del batch, che tende a +1 in caso di collasso. La seconda è la dispersione media per dimensione dell'embedding, che nello stesso caso tende a zero. Un valore del primo indicatore superiore a 0,9 genera una segnalazione esplicita nel log insieme alla loss e al learning rate corrente, e le stesse quantità vengono tracciate come serie temporali quando il monitoraggio remoto è attivo.

Il comportamento osservato è quello desiderato, ma con un dettaglio che vale la pena riportare. Nelle primissime epoche il coseno fuori diagonale sale, e nel caso di GINECONV con NT-Xent tocca 0,52 alla prima epoca, quando la rete non ha ancora imparato a distinguere e assegna a tutte le scene vettori simili. È il collasso nella sua forma iniziale, e viene riassorbito entro poche epoche. Da quel punto in avanti il valore resta nell'intorno dello zero per tutte e trecento le epoche in ogni esecuzione, come mostra il pannello inferiore di figura 8, e la soglia di segnalazione non viene mai avvicinata.

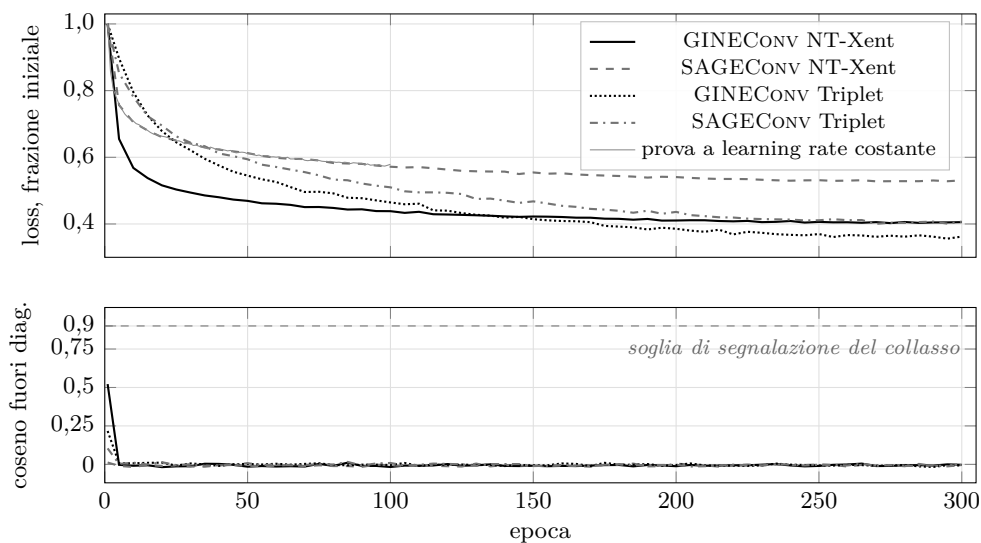


Figura 8: Andamento del training nelle quattro esecuzioni definitive, con gli stessi tratti nei due pannelli. In alto la loss espressa come frazione del proprio valore iniziale, scelta necessaria perché Triplet e NT-Xent vivono su scale diverse e i loro valori assoluti non sono confrontabili tra loro, mentre le forme di convergenza lo sono; in valore assoluto le due esecuzioni NT-Xent terminano a 2,05 e 2,10, le due Triplet a 0,11 entrambe. La linea sottile è la prova preliminare a learning rate costante, interrotta a cento epoche mentre era ancora in discesa. In basso il coseno medio tra elementi diversi del batch, che dopo un transitorio nelle prime epoche resta prossimo allo zero per tutta la durata, lontano dalla soglia oltre la quale il codice segnala il collasso.

Leggere le relazioni incide infine poco sulla durata dell'addestramento, dato che GINECONV chiude in 10'44" contro i 10'29" di SAGECONV sotto NT-Xent, e in 11'18" contro 10'31"

sotto Triplet. Il sovrapprezzo del ramo relazionale si colloca quindi tra il due e il sette per cento del tempo totale, per cui l’eventuale guadagno misurato nelle ablation non va soppesato contro un costo significativo.

4 Valutazione sperimentale e analisi dei risultati

Questo capitolo presenta l’intera campagna sperimentale volta a quantificare l’efficacia del retrieval basato su grafi di scena e arricchimento ontologico, dal protocollo di ablation alla valutazione sulle due scale, fino ai test zero-shot e alle indagini di robustezza e interpretabilità.

4.1 Protocollo sperimentale e disegno di ablation

Le scelte descritte nelle sezioni precedenti — l’operatore di convoluzione, la loss, l’origine dei positivi — sono tutte discutibili in astratto, e ciascuna è stata quindi trasformata in un confronto misurabile. Il principio seguito è che nessuna affermazione del lavoro poggi su una singola esecuzione, ma sempre sulla differenza tra due configurazioni che si distinguono per una sola variabile.

4.1.1 Le esecuzioni principali

Le due variabili principali sono l’architettura e la loss, e sono state provate in tutte e quattro le combinazioni possibili (tabella 4). Il confronto tra GINECONV e SAGECONV è quello centrale del lavoro, perché se l’informazione relazionale non producesse alcun guadagno l’intera scelta di rappresentare le scene come grafi resterebbe senza sostegno sperimentale. Provarlo sotto entrambe le loss fa sì che ogni domanda riceva due risposte invece di una. Se GINECONV batte SAGECONV sotto NT-Xent e lo batte di nuovo sotto Triplet, il merito è dell’architettura; se lo batte in un caso solo, il vantaggio dipende anche dalla loss con cui è stato addestrato, ed è una conclusione diversa. Quattro esecuzioni bastano a distinguere le due situazioni, tre no.

Tabella 4: Le quattro esecuzioni principali e i rispettivi checkpoint. Gli iperparametri sono identici in tutte le celle, che differiscono soltanto per le due variabili sotto esame.

	NT-Xent	Triplet Margin
SAGECONV (NoRel)	<code>gcn_sage</code>	<code>gcn_sage_triplet</code>
GINECONV (Rel)	<code>gcn_gine</code>	<code>gcn_gine_triplet</code>

Tutto ciò che non è sotto esame viene congelato ai valori raccolti in tabella 12. La suddivisione del corpus è generata una volta sola con seed fisso e salvata su file, quindi tutte le esecuzioni leggono letteralmente le stesse scene negli stessi insiemi. Sul piano operativo la griglia è realizzata da un unico script di job che riceve l’architettura e la loss come argomenti, per cui le celle della tabella corrispondono ad altrettante invocazioni dello stesso comando e non a varianti di codice.

Un limite di questo disegno va dichiarato. L’ordine con cui le scene vengono raccolte nei batch non è fissato da un seed, e nessuna configurazione è stata ripetuta più volte per stimare la

variabilità dovuta a questo fattore. Le differenze ampie tra le celle sono quindi interpretabili, mentre scarti dell'ordine del punto percentuale su una singola metrica non vanno letti come evidenza di superiorità.

Le esecuzioni descritte in questa griglia si focalizzano intenzionalmente sulle sole varianti architetturelle e di loss, operando sul sub-set di base privo di arricchimento ontologico. Questa scelta progettuale permette di mantenere rigorosamente congelati gli iperparametri per le quattro esecuzioni principali, isolando così in purezza il contributo della rete e della loss. L'impatto delle ulteriori variabili sperimentali, ovvero l'arricchimento semantico delle macro-categorie prodotto da Virtuoso (sezione 2.2) e l'addestramento sull'intero corpus di dati (*full-set*), viene introdotto e quantificato progressivamente nei paragrafi successivi.

4.1.2 Circolarità della ground truth e contromisure

Resta da affrontare il limite dichiarato in sezione 2.3.6, ovvero il fatto che un metro derivato dai grafi favorisca per costruzione i modelli che leggono i grafi. Eliminarlo richiederebbe una ground truth di origine indipendente che per questo corpus non esiste, ma lo si può circoscrivere in modo che le conclusioni non ne dipendano.

La prima contromisura è strutturale e riguarda quali confronti vengono usati per argomentare. Le ablation interne mettono a confronto modelli che leggono *tutti* i grafi e sono quindi esposti allo stesso vantaggio sistematico, il quale si elide nella differenza tra loro. L'affermazione che le relazioni aiutino non poggia perciò sul confronto con le baseline visive, dove il vantaggio strutturale agirebbe indisturbato, ma sul confronto tra due encoder a grafo che ricevono lo stesso input e differiscono soltanto per la capacità di leggere le etichette degli archi.

La seconda è la calibrazione del criterio con un giudice esterno puramente visivo (sezione 2.3.5), che non sa nulla dei grafi e fornisce una stima indipendente della quota di rumore nei positivi. La terza è la valutazione zero-shot, dove il modello viene interrogato su scene appartenenti a una distribuzione diversa da quella di training e non può quindi beneficiare di regolarità specifiche del corpus su cui è stato addestrato. Il confronto con le baseline visive resta nella relazione, ma va letto con la premessa esplicita che il metro non è neutrale rispetto ai metodi confrontati.

4.2 Valutazione del retrieval sul sotto-insieme controllato (15k)

In questa sezione viene descritto il modulo di retrieval vettoriale e vengono analizzati i risultati ottenuti sul sotto-insieme controllato di validazione (~ 15.000 scene, con 771 query di test).

4.2.1 Infrastruttura di ricerca vettoriale: FAISS

Il motore di ricerca per similarità è implementato tramite la libreria ad alte prestazioni FAISS (*Facebook AI Similarity Search*) [1]. Per garantire una ricerca esatta senza approssimazioni o quantizzazioni distorcenti, è stato impiegato l'indice `IndexFlatIP` (*Inner Product*). Poiché tutti i vettori di embedding prodotti dagli encoder (sia GCN sia le baseline visive) vengono rigorosamente normalizzati in norma ℓ_2 prima dell'indicizzazione ($\|\mathbf{z}\|_2 = 1$), il prodotto

scalare calcolato da FAISS coincide esattamente con la similarità coseno:

$$\langle \mathbf{z}_q, \mathbf{z}_g \rangle = \frac{\mathbf{z}_q \cdot \mathbf{z}_g}{\|\mathbf{z}_q\|_2 \|\mathbf{z}_g\|_2} = \cos(\theta) \quad (2)$$

L’indicizzazione e l’interrogazione avvengono in memoria primaria, garantendo latenze di ricerca dell’ordine dei millisecondi sull’intera galleria di test.

4.2.2 Metriche di valutazione del Ranking

Data una query q , sia $\mathcal{R}_k(q)$ l’insieme dei primi k elementi recuperati dalla galleria e sia $\mathcal{G}(q)$ l’insieme dei positivi di *ground truth* definiti dal criterio ibrido (sezione 2.3). Le prestazioni del sistema vengono quantificate attraverso tre metriche standard:

- **Precision@K:** Frazione dei primi k risultati recuperati che risultano semanticamente rilevanti:

$$\text{Precision@k}(q) = \frac{|\mathcal{R}_k(q) \cap \mathcal{G}(q)|}{k} \quad (3)$$

- **Recall@K:** Frazione dei positivi totali della query recuperati all’interno della Top- k :

$$\text{Recall@k}(q) = \frac{|\mathcal{R}_k(q) \cap \mathcal{G}(q)|}{|\mathcal{G}(q)|} \quad (4)$$

- **Accuracy@K (Hit Rate@K):** Metrica binaria che indica se *almeno un* elemento rilevante è presente tra i primi k risultati restituiti dal motore di ricerca:

$$\text{Accuracy@k}(q) = \mathbb{I}(|\mathcal{R}_k(q) \cap \mathcal{G}(q)| \geq 1) \quad (5)$$

4.2.3 Analisi quantitativa dei modelli a grafo sul Subset (15k)

La tabella 5 raccoglie i risultati di retrieval ottenuti su tutte le combinazioni architetturali e di loss sul sotto-insieme controllato (15k), mettendo a confronto la configurazione con arricchimento ontologico (*Semantic Web*) e quella nativa (*Baseline*).

Tabella 5: Prestazioni di retrieval sul sotto-insieme controllato (Subset 15k, 771 query) al variare dell’arricchimento ontologico, dell’architettura e della loss.

Configurazione	Architettura	Loss	P@1 (%)	P@5 (%)	P@10 (%)	R@10 (%)	Acc@10 (%)	Acc@20 (%)
Semantic Web	SAGECONV	NT-Xent	35,80	20,57	13,85	35,23	68,61	76,65
	GINECONV	NT-Xent	38,39	21,74	14,51	36,38	70,04	78,34
	SAGECONV	Triplet	44,75	23,53	15,16	38,29	71,98	79,90
	GINECONV	Triplet	47,34	24,85	15,71	39,85	74,45	81,58
GQA Nativo	SAGECONV	NT-Xent	29,70	16,78	11,35	28,88	58,11	64,46
	GINECONV	NT-Xent	31,91	17,51	11,79	30,04	60,18	67,06
	SAGECONV	Triplet	38,26	18,86	12,39	31,63	62,78	67,70
	GINECONV	Triplet	40,08	19,74	12,75	32,48	63,55	68,35

I dati evidenziano con chiarezza tre evidenze sperimentali:

1. **Impatto della Convoluzione Relazionale (GINEConv vs SAGEConv):** L'encoder edge-aware GINEConv supera sistematicamente SAGEConv in tutte le metriche. Sotto Triplet Loss con arricchimento semantico, GINEConv ottiene una Top-1 Precision del 47,34% rispetto al 44,75% di SAGEConv (+2,59% assoluto), e raggiunge un'Accuracy@20 dell'81,58%. Questo conferma che la capacità di codificare le relazioni d'azione produce uno spazio metrico significativamente più discriminativo.
2. **Efficacia della Funzione di Perdita (Triplet vs NT-Xent):** La funzione di perdita Triplet Margin con semi-hard mining distacca nettamente la formulazione NT-Xent (InfoNCE), portando la Top-1 Precision dal 38,39% al 47,34% (+8,95% assoluto su GINEConv). Il vincolo locale con margine geometrico si dimostra molto più idoneo a separare positivi semantici complessi rispetto alla normalizzazione globale softmax.
3. **Guadagno dell'Arricchimento Ontologico:** L'espansione dei nodi tramite le macro-categorie inferite da Virtuoso apporta un incremento determinante su tutta la linea: il modello `gcn_gine_triplet` arricchito supera la controparte nativa di +7,26% assoluto su Top-1 Precision (47,34% vs 40,08%) e di +13,23% assoluto su Accuracy@20 (81,58% vs 68,35%), dimostrando il valore delle tecnologie del Semantic Web nel regolarizzare e arricchire l'apprendimento profondo.

4.2.4 Confronto con le baseline visive sul Subset (15k)

La tabella 6 pone a diretto confronto il modello a grafo vincente (`gcn_gine_triplet`) con le baseline puramente visive non strutturate (RESNET e CLIP).

Tabella 6: Confronto sul Subset (15k) tra il miglior modello GCN e le baseline visive non strutturate.

Modello	P@1 (%)	P@5 (%)	P@10 (%)	R@10 (%)	Acc@10 (%)	Acc@20 (%)
ResNet-50 (Pixel)	23,73	10,04	6,38	16,26	36,96	44,88
CLIP ViT-B/32 (Vision-Lang)	23,73	10,61	6,86	17,59	37,35	44,36
GCN GINE Triplet (Grafo + Onto)	47,34	24,85	15,71	39,85	74,45	81,58

Il modello basato su grafo stacca nettamente le feature basate su soli pixel: il margine su Accuracy@20 rispetto a CLIP è di ben +37,22% (81,58% contro 44,36%), e la Top-1 Precision risulta esattamente raddoppiata (47,34% contro 23,73%). Questo divario attesta che l'esplicitazione della struttura topologica e dei predicati relazionali consente di recuperare scene composizionali con una precisione irraggiungibile per encoder visivi globali.

4.3 Scala del dataset (55k) e test zero-shot incrociati

Mentre nella sezione precedente l'analisi del retrieval e l'impatto dell'arricchimento ontologico sono stati quantificati sul sotto-insieme controllato di 15k scene, questo paragrafo affronta il problema fondamentale del *salto di scala* e della *generalizzazione out-of-distribution*.

Tutte le esecuzioni sul *full-set* hanno mantenuto rigorosamente congelati gli stessi iperparametri definiti sul *sub-set* (tabella 4), garantendo che ogni variazione misurata sia ascrivibile

esclusivamente alla dimensione del corpus e alla presenza di distrattori.

4.3.1 Effetto dell’ampliamento della galleria sul Full-Set (55k)

All’aumentare delle dimensioni del dataset, la galleria di test cresce da circa 3.500 scene nel sotto-insieme a oltre 20.000 nel *full-set*. Questo ampliamento introduce migliaia di *distrattori* — scene visivamente o semanticamente simili che competono per le prime posizioni del ranking.

La tabella 7 riporta le prestazioni aggregate sul *full-set* per tutti i modelli addestrati e per le baseline visive.

Tabella 7: Benchmark completo sul Full-Set (oltre 55.000 grafi, 1000 queries) al variare dell’arricchimento ontologico, dell’architettura e della loss.

Configurazione	Architettura	Loss	P@1 (%)	P@5 (%)	P@10 (%)	R@10 (%)	Acc@10 (%)	Acc@20 (%)
Semantic Web	SAGECONV	NT-Xent	26,80	13,50	8,59	20,73	49,30	58,10
	GINECONV	NT-Xent	28,70	14,56	9,39	22,52	52,40	61,50
	SAGECONV	Triplet	34,70	16,32	10,44	25,17	56,30	65,10
	GINECONV	Triplet	36,90	17,28	10,95	26,41	58,80	67,20
GQA Nativo	SAGECONV	NT-Xent	26,30	13,10	8,46	20,39	48,50	58,10
	GINECONV	NT-Xent	28,50	14,46	9,40	22,68	51,70	61,30
	SAGECONV	Triplet	34,90	16,58	10,46	25,25	55,30	65,20
	GINECONV	Triplet	37,30	17,34	10,98	26,46	59,10	67,10
ResNet-50 (Pixel)		—	16,60	6,86	4,28	10,33	29,00	37,00
CLIP ViT-B/32 (Vision-Lang)		—	20,00	8,48	5,34	12,90	32,20	41,20

Come teoricamente atteso, l’aumento della dimensione della galleria causa una flessione fisiologica controllata su tutte le metriche assolute: l’Accuracy@20 del modello migliore (**gcn_gine_triplet**) passa dall’81,58% al 67,20% (−14,38%). Tuttavia, è di fondamentale importanza osservare che:

- La gerarchia relativa tra i modelli rimane perfettamente invariata: GINECONV supera SAGECONV e la Triplet Margin supera sistematicamente NT-Xent.
- Il vantaggio competitivo sulle baseline visive si riconferma netto e indiscutibile: **gcn_gine_triplet** supera ResNet-50 di +30,20% (67,20% vs 37,00%) e CLIP di +26,00% (67,20% vs 41,20%) ad Accuracy@20.

4.3.2 Valutazione Incrociata (Cross-Evaluation Zero-Shot)

Per testare la capacità di generalizzazione e verificare se gli encoder abbiano appreso rappresentazioni invarianti e trasferibili, sono stati condotti esperimenti di valutazione incrociata *zero-shot* in due direzioni complementari.

1. Forward Cross-Evaluation (Modelli 55k testati su Galleria 15k): In questo scenario, i modelli addestrati sul corpus completo vengono impiegati per interrogare la galleria del sotto-insieme controllato da 15k (tabella 8).

Tabella 8: Forward Cross-Evaluation: Prestazioni dei modelli addestrati sul Full-Set (55k) valutati sul test-set da 15k.

Configurazione	Architettura	Loss	P@1 (%)	P@5 (%)	P@10 (%)	R@10 (%)	Acc@10 (%)	Acc@20 (%)
Semantic Web	SAGECONV	NT-Xent	39,43	21,74	14,51	36,37	70,43	78,86
	GINECONV	NT-Xent	41,50	22,67	14,98	37,86	70,82	78,99
	SAGECONV	Triplet	46,95	24,10	15,47	39,09	72,89	80,42
	GINECONV	Triplet	49,68	25,08	16,02	40,50	74,71	81,71
GQA Nativo	SAGECONV	NT-Xent	26,85	14,58	9,77	24,87	50,06	58,11
	GINECONV	NT-Xent	27,50	15,05	10,09	25,69	52,40	60,18
	SAGECONV	Triplet	29,70	15,64	10,57	27,04	54,73	62,78
	GINECONV	Triplet	31,78	16,45	10,93	27,94	56,68	63,55

Il modello arricchito semanticamente `gcn_gine_triplet` (55k) raggiunge prestazioni straordinarie sul dominio 15k, toccando il picco assoluto di 81,71% di Accuracy@20 e il 49,68% di Top-1 Precision, superando persino il modello addestrato nativamente sul 15k. Questo evidenzia che l’addestramento su larga scala conferisce all’encoder una rappresentazione geometrica ancora più ricca e precisa.

Al contempo, il confronto tra blocco *Semantic Web* e blocco *GQA Nativo* in tabella 8 svela un fenomeno teorico fondamentale: il modello nativo (senza ontologia) subisce un calo sensibile (63,55% vs 81,71%). Tale divario documenta l’effetto del *feature distribution mismatch*: la galleria 15k include le macro-categorie ontologiche; l’encoder arricchito sfrutta pienamente questa informazione allineata, mentre l’encoder nativo sperimenta uno shift distributivo nei vettori dei nodi.

2. Reverse Cross-Evaluation (Modelli 15k testati su Galleria 55k): Nel percorso inverso (tabella 9), i modelli addestrati sul sotto-insieme ridotto vengono chiamati a indicizzare e ricercare nell’ampia galleria da 55k (con oltre 20.000 distrattori).

Tabella 9: Reverse Cross-Evaluation: Prestazioni dei modelli addestrati sul Subset (15k) valutati sul test-set completo da 55k.

Configurazione	Architettura	Loss	P@1 (%)	P@5 (%)	P@10 (%)	R@10 (%)	Acc@10 (%)	Acc@20 (%)
Semantic Web	SAGECONV	NT-Xent	13,40	6,86	4,46	10,76	28,70	38,20
	GINECONV	NT-Xent	15,10	7,52	4,87	11,72	30,70	40,50
	SAGECONV	Triplet	17,50	8,78	5,63	13,57	34,70	44,80
	GINECONV	Triplet	18,90	9,74	6,40	15,42	38,50	49,70

In questo scenario si osserva un calo marcato: l’Accuracy@20 scende al 49,70% e la Top-1 Precision al 18,90%. Questo esperimento dimostra quantitativamente la necessità dell’addestramento su larga scala sul cluster HPC: un modello addestrato su sole 15.000 scene non ha esplorato una varietà sufficiente di triple e configurazioni relazionali per disambiguare l’enorme spazio geometrico generato da oltre 20.000 distrattori, confermando l’importanza scientifica di scalare l’addestramento all’intero corpus di 55k grafi.

4.4 Robustezza strutturale e interpretabilità

Per completare la validazione dell'architettura proposta sono state condotte due indagini avanzate sul solo modello vincente, `gcn_gine_triplet` (Semantic Web), sulla resilienza a scene incomplete e sull'interpretabilità delle decisioni tramite *GNNEExplainer* [2].

4.4.1 Test di robustezza a perturbazioni strutturali (Self-Retrieval)

Nei contesti applicativi reali, i grafi estratti da algoritmi di scene graph generation possono presentare omissioni di oggetti o archi spuri dovuti a occlusioni o limitazioni dei detector visivi. Per quantificare la capacità del modello di operare in condizioni degradate, è stato progettato un protocollo di **Self-Retrieval**:

- Viene selezionato un campione statistico di 1.000 scene estratte dalla galleria di test.
- Ciascun grafo viene progressivamente perturbato applicando un tasso di *node dropout* casuale $p \in \{0,0,0,1,0,2,0,3,0,4\}$, corrispondente alla distruzione controllata dal 10% fino al 40% dei nodi totali.
- La rimozione dei nodi induce il ricalcolo coerente del sottografo tramite l'operatore `subgraph` di PyTorch Geometric, preservando esclusivamente gli archi che connettono nodi superstiti e garantendo che nessun grafo degeneri a zero elementi.
- I grafi perturbati vengono rielaborati dall'encoder `gcn_gine_triplet` e proiettati come query nel motore FAISS, verificando se il vettore degradato sia ancora in grado di rintracciare la versione integra originale della propria scena all'interno della galleria.

I risultati sperimentali sono riportati in tabella 10 e visualizzati graficamente in figura 9.

Tabella 10: Degrado delle prestazioni di Self-Retrieval su 1.000 scene al variare della percentuale di nodi rimossi per il modello `gcn_gine_triplet`.

Percentuale Nodi Rimossi (p)	0%	10%	20%	30%	40%
Accuracy@20 / HitRate (%)	99,80	99,20	97,50	95,10	87,20
Precision@1 (%)	99,60	95,60	89,80	79,80	64,80

Dall'andamento delle curve emergono considerazioni di rilievo:

- **Resilienza Morbida (Non-Lineare):** Fino al 20% di nodi distrutti, la perdita di accuratezza ad @20 è pressoché trascurabile (scendendo da 99,80% a 97,50%), con la Top-1 Precision che si mantiene a un ragguardevole 89,80%.
- **Tenuta a Forti Perturbazioni (40%):** Anche rimuovendo quasi la metà della scena (40%), il sistema riesce a identificare l'identità corretta nella Top-20 nell'87,20% dei casi e al primissimo posto nel 64,80%.

Questo comportamento dimostra che la GCN con convoluzione GINECONV non si limita a memorizzare configurazioni locali fragili, ma apprende una rappresentazione semantica globale e distribuita: l'aggregazione di messaggi lungo i percorsi superstiti permette al pooling

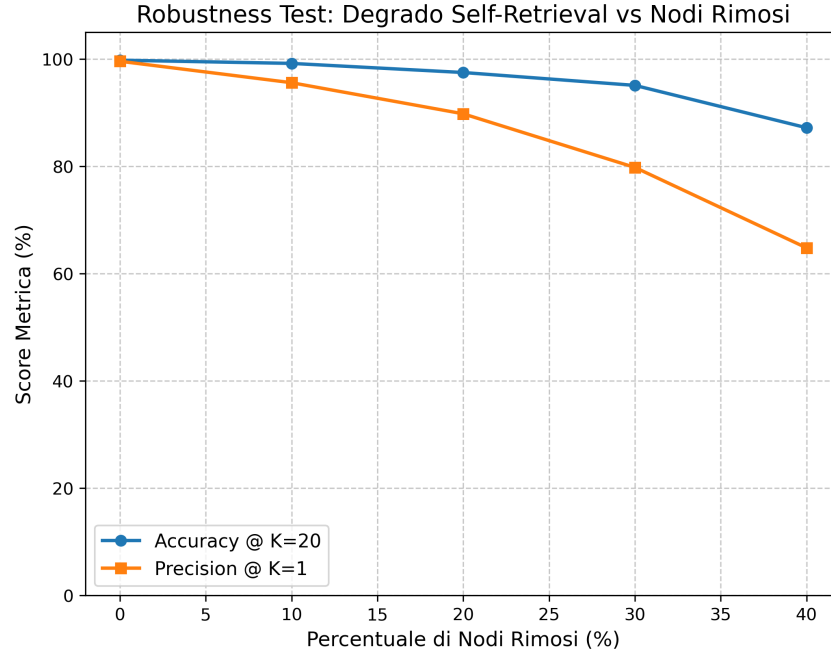


Figura 9: Curva di degradazione delle metriche di Self-Retrieval (Accuracy@20 e Precision@1) al crescere della percentuale di perturbazione strutturale dei nodi.

finale di conservare l'essenza concettuale della scena anche in presenza di significative lacune informative.

4.4.2 Interpretabilità delle decisioni tramite GNNExplainer

Per superare la natura di “scatola nera” della rete neurale e verificare che le similarità calcolate riflettano proprietà concettuali sensate, è stato integrato il framework **GNNExplainer** [2].

Poiché l'encoder non compie una classificazione discreta ma proietta la scena in uno spazio metrico continuo a 256 dimensioni, GNNEXPLAINER è stato configurato in modalità *regression* a livello di grafo. L'algoritmo ottimizza congiuntamente una maschera sui nodi $\mathbf{M}_V \in [0, 1]^{|V|}$ e una maschera sugli archi $\mathbf{M}_E \in [0, 1]^{|E|}$ per massimizzare la concordanza tra la proiezione del grafo mascherato e l'embedding target originale $\mathbf{z}$, penalizzando l'entropia delle maschere per isolare una sottostruttura compatta e massimamente esplicativa.

In figura 10 viene illustrata l'applicazione dell'explainer a una complessa scena d'azione di GQA (ID 2363112, composta da 11 nodi e 21 archi), affiancando la fotografia originale di un surfista alla mappa di salienza dello scene graph corrispondente.

L'analisi quantitativa della maschera di salienza rivela una gerarchia concettuale rigorosa e perfettamente coerente con la semantica della scena:

1. **Entità Focali Primarie:** I nodi che ricevono i coefficienti di salienza più elevati ($> 0,085$) corrispondono agli elementi cardine del contesto narrativo e dell'interazione sportiva: *Surfboard* (0,087), *Man* (0,085), *Arm* (0,086) e *Hair* (0,093), dimostrando che

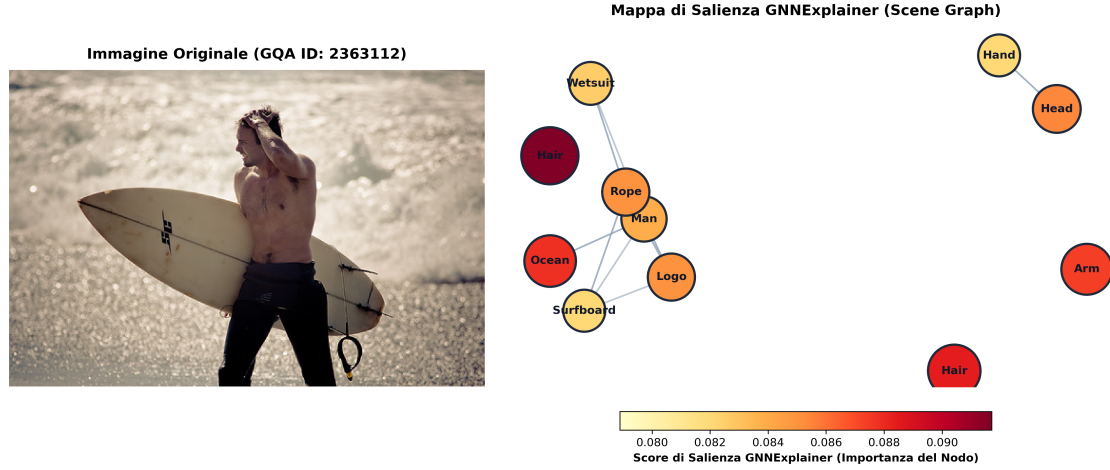


Figura 10: Confronto visivo tra la fotografia originale (a sinistra, GQA ID: 2363112) e la mappa di salienza estratta da GNNExplainer sullo scene graph corrispondente (a destra). L'intensità del colore rosso e la dimensione dei nodi sono direttamente proporzionali all'importanza causale nell'orientare l'embedding latente della GCN.

la GCN identifica la persona e l'attrezzatura d'azione come fulcro della rappresentazione semantica.

2. **Filtro del Rumore Periferico:** Elementi accessori, nodi secondari o attributi marginali (*Rope*, etichette di sfondo) ricevono coefficienti di salienza inferiori, contribuendo solo marginalmente all'orientamento del vettore metrico.

Questo risultato conferma che la rete `gcn_gine_triplet` ha appreso ad allocare l'attenzione dello spazio latente sugli oggetti dominanti e sulle loro interazioni chiave, conferendo piena trasparenza e interpretabilità geometrica alle decisioni di retrieval.

5 Estensioni di sistema e deployment

Questa sezione documenta le estensioni avanzate di ingegnerizzazione del software realizzate a corredo del motore di retrieval: l'integrazione di un modello *Vision-Language* per l'estrazione dinamica di grafi da immagini arbitrarie dell'utente, e l'architettura a microservizi asincrona distribuita per il deployment locale e cloud.

5.1 Estrazione dinamica di scene graph tramite VLM

Finora, il sistema ha operato assumendo che la query fosse uno scene graph di *ground truth* già disponibile nel dataset GQA. Per la realizzazione della Demo interattiva finale, è stato introdotto un approccio molto più sfidante: estrarre un grafo dinamico a partire da un'immagine qualsiasi caricata dall'utente. Per assolvere a questo compito, è stato integrato un modello *Vision-Language* (VLM), nello specifico **Qwen2.5-VL-7B**, incaricato di analizzare la fotografia ed estrarre oggetti, attributi e relazioni sotto forma di dati strutturati. Dato il notevole carico computazionale richiesto per l'inferenza del VLM, questo *dynamic extractor* viene eseguito direttamente sui nodi GPU del cluster HPC dell'Università di Catania (UNICT).

5.1.1 Il problema dell'allineamento a vocabolario aperto

A differenza della ground truth di GQA, costruita su un vocabolario chiuso e normalizzato, un modello generativo produce testo libero. Un nodo che il VLM descrive come *guy*, dove il sistema di training ha sempre visto *man*, provocherebbe uno shift semantico a valle, spingendo l'embedding vettoriale di Nomic fuori dalla distribuzione su cui la rete GCN è stata addestrata.

Per mitigare questo problema, si è adottato un approccio ibrido:

1. **Constrained Prompting:** il VLM viene interrogato tramite prompt rigorosi per forzare l'output in un formato JSON predefinito, limitando il raggio dei sinonimi ed evitando testo colloquiale.
2. **Arricchimento Ontologico Deterministico:** i termini estratti vengono interfacciati con l'ontologia ospitata su Virtuoso. Invece di chiedere le super-classi e le astrazioni al modello generativo (rischiando gravi allucinazioni semantiche), il sistema interroga l'ontologia per estrarre le macro-categorie esatte corrispondenti ai nodi identificati dal VLM.

5.1.2 Ricostruzione della rappresentazione

Durante il processo di inferenza della Demo, gli identificativi univoci dei nodi, non possedendo un ground truth associato, vengono sostituiti da ID fittizi temporanei. Ciò che risulta cruciale è la ricostruzione della stringa canonica: concatenando il nome dell'oggetto estratto dal VLM con i suoi attributi visivi e la relativa macro-categoria ricavata da Virtuoso, il sistema garantisce che la stringa finale iniettata nell'embedder testuale NOMIC sia formalmente e sintatticamente identica a quelle utilizzate durante la fase di addestramento.

5.1.3 Prototipo funzionante e limiti architetturali

Allo stato attuale, l'estrattore dinamico è stato integrato con successo nella pipeline della Demo come prototipo interamente funzionante. Le esecuzioni documentano una buona stabilità su immagini semplici e con pochi oggetti dominanti, permettendo al gateway FastAPI e alla successiva ricerca vettoriale su FAISS di recuperare scene pertinenti dal full-set.

Permangono tuttavia limiti fisiologici sulle scene altamente affollate, dove il VLM tende a generare JSON troppo ramificati o ad allucinare relazioni spaziali inesistenti. Un futuro raffinamento ingegneristico potrebbe includere moduli di *saliency filtering* per scartare nodi irrilevanti e semplificare il grafo prima della codifica, stabilizzando ulteriormente l'input fornito all'encoder a grafo.

5.2 Infrastruttura distribuita e deployment

Questa sezione documenta l'architettura di deployment del progetto, progettata per esporre il motore di retrieval sviluppato attraverso un'infrastruttura moderna, scalabile e distribuita, sia in ambiente locale che in ottica Cloud.

5.2.1 Architettura a Microservizi e Pipeline

Il sistema finale è stato ingegnerizzato seguendo un pattern a microservizi asincroni, per garantire disaccoppiamento e scalabilità. Il punto di ingresso per il traffico web (sia per la dashboard utente che per i client REST) è gestito da un gateway **FastAPI**. Quando un utente carica una nuova immagine per la ricerca, la richiesta non blocca il server: il gateway inserisce un job in una coda di messaggistica (**RabbitMQ** nell'ambiente locale, **Amazon SQS** nel cloud) e restituisce un identificativo al client.

Un nodo specializzato, l'*AI Worker*, consuma asincronamente i job in coda. Il cuore di questo worker è una complessa pipeline di elaborazione divisa in tre fasi:

1. **Estrazione dinamica del grafo:** L'immagine viene processata dal *Vision-Language Model Qwen2.5-VL-7B*, che viene interrogato per restituire un JSON strutturato contenente gli oggetti, i loro attributi e le relazioni spaziali.
2. **Embedding vettoriale:** I nodi testuali estratti, arricchiti con le macro-categorie di Virtuoso, vengono convertiti in vettori densi a 768 dimensioni tramite l'embedder testuale **Nomic**.
3. **Inferenza e Ricerca:** Il grafo risultante viene processato dalla **GCN** (GINECONV) addestrata. L'embedding finale del grafo viene quindi confrontato con l'intero *full-set* tramite calcolo della similarità coseno su un indice vettoriale **FAISS** ad altissime prestazioni.

Per minimizzare i tempi di risposta ed evitare calcoli ridondanti, un database in-memory **Redis** funge da livello di *caching*, mentre un database relazionale **PostgreSQL** è adibito alla persistenza dei log delle query e dello storico degli utenti.

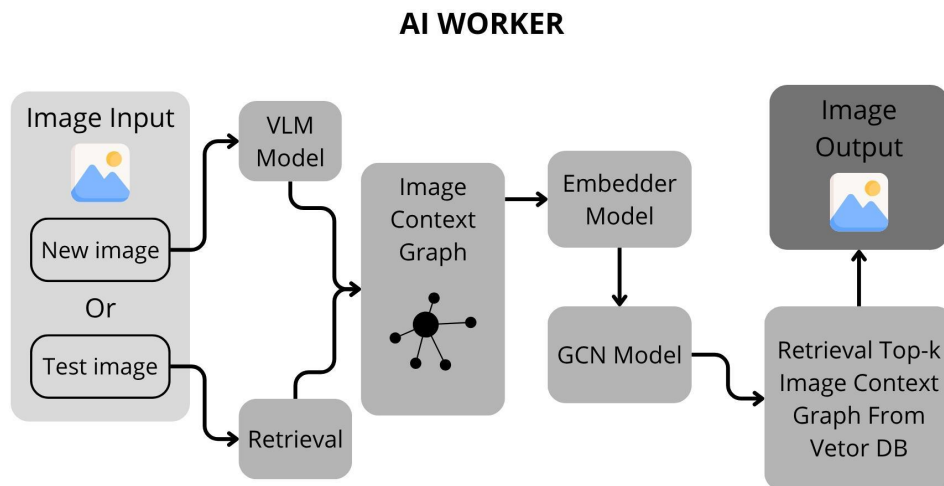


Figura 11: Pipeline dell'AI Worker: elaborazione dell'immagine tramite VLM, estrazione del grafo di contesto, embedding e retrieve dal Vector DB.

5.2.2 Orchestrazione Locale e Cloud IaC

Il deployment del progetto è stato concepito in due fasi complementari. La fase di sviluppo, validazione e l'esecuzione della Demo interattiva si basano su un cluster locale orchestrato tramite **K3s** (una distribuzione Kubernetes certificata e ottimizzata). In questo ambiente, ogni componente (FastAPI, RabbitMQ, PostgreSQL, Redis e l'AI Worker) è containerizzato tramite **Docker**. Questa architettura ha permesso di isolare le dipendenze, parallelizzare i processi e garantire un'alta disponibilità simulando a tutti gli effetti un ambiente di produzione locale.

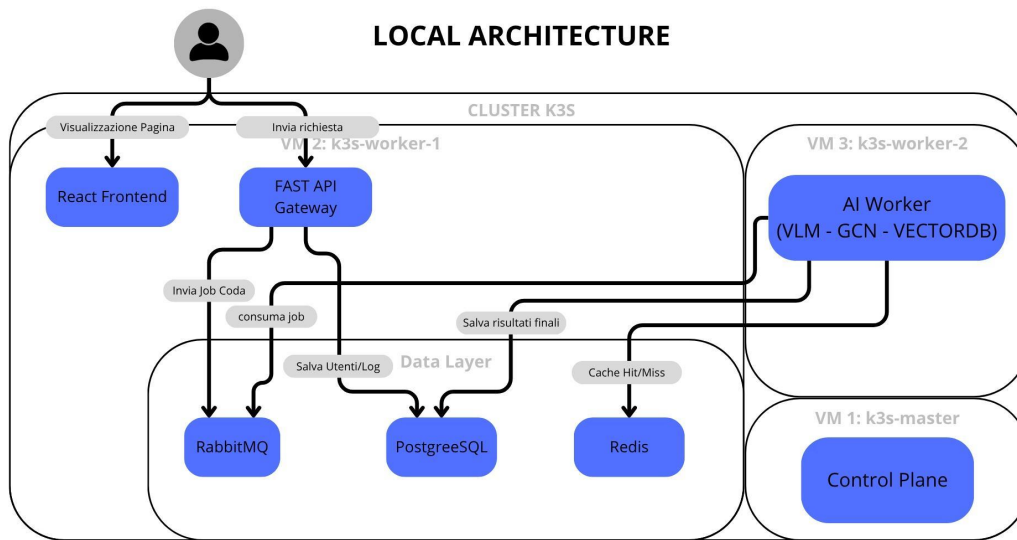


Figura 12: Architettura locale basata su cluster K3s, con gateway FastAPI, code RabbitMQ e Data Layer (PostgreSQL e Redis).

Parallelamente, al fine di dimostrare la prontezza del sistema per un rilascio *enterprise* distribuito, è stata implementata un'infrastruttura *Infrastructure-as-Code* (IaC) tramite **Terraform** per automatizzare il porting su **AWS**. Il codice Terraform definisce programmaticamente una **VPC** isolata con subnet pubbliche e private. Il traffico in ingresso è bilanciato da un *Application Load Balancer* (ALB) verso i nodi Kubernetes ospitati su istanze **EC2**. Per abbattere l'overhead operativo nel cloud, i servizi accessori sono stati delegati a soluzioni AWS completamente gestite: le immagini caricate dagli utenti vengono salvate su **S3**, la coda dei task asincroni si appoggia ad **SQS**, e il Data Layer sfrutta **RDS** per PostgreSQL e **ElastiCache** per Redis.

6 Discussione e conclusioni

Questo capitolo finale offre una sintesi interpretativa dei risultati conseguiti, formalizza le ragioni teoriche alla base dei comportamenti osservati, discute con trasparenza scientifica i limiti metodologici e traccia le prospettive di evoluzione futura del lavoro.

do nodi lessicalmente disgiunti ma semanticamente affini (es. *cat* e *dog* sotto **Animale**) in porzioni contigue dello spazio prima ancora del passaggio nella rete.

6.1.2 Perché la Triplet Margin Loss supera sistematicamente NT-Xent

La netta superiorità di Triplet Margin rispetto a NT-Xent (InfoNCE) merita una spiegazione formale legata alla natura dei dati di addestramento:

- **La Severità della Softmax in NT-Xent:** La loss NT-Xent applica una normalizzazione esponenziale su tutti gli elementi del batch, trattando ogni coppia non-anchor/positivo come un negativo tassativo da respingere. In un dataset ad alta densità compositiva come GQA, è estremamente probabile che all'interno di un batch da 128 elementi compaiano scene parzialmente affini (falsi negativi non etichettati). L'esponenziale a denominatore genera gradienti repulsivi elevati contro queste scene affini, inducendo una frammentazione eccessiva dello spazio latente (*over-clustering*) e contrastando la convergenza su concetti condivisi.
- **La Tolleranza Locale del Margine:** La Triplet Loss con semi-hard mining impone un vincolo geometrico puramente locale: richiede unicamente che la distanza tra anchor e positivo sia inferiore a quella con il negativo di almeno un margine α :

$$\mathcal{L}_{\text{triplet}} = \max(0, D(\mathbf{z}_a, \mathbf{z}_p) - D(\mathbf{z}_a, \mathbf{z}_n) + \alpha) \quad (6)$$

Quando la condizione è soddisfatta, il gradiente si annulla esattamente ($\nabla = 0$). Questa proprietà di saturazione conferisce alla Triplet Loss una naturale tolleranza al rumore intrinseco dei positivi estratti dal mining euristico, evitando di penalizzare le anchor rispetto a negativi casuali moderatamente simili.

6.1.3 Limiti metodologici e minacce alla validità

Un'onesta valutazione accademica richiede la dichiarazione esplicita dei limiti progettuali:

- **Circolarità Parziale della Ground Truth:** Il criterio di somiglianza (sezione 2.3) è rule-based e opera sulla struttura dei grafi. Sebbene indispensabile per garantire determinismo e riproducibilità, tale impostazione favorisce strutturalmente i modelli basati su grafo rispetto alle baseline puramente visive (CLIP e ResNet-50). Di conseguenza, il divario di performance (+26–37%) non va interpretato come un fallimento di CLIP (che eccelle nella semantica generale zero-shot), ma come la quantificazione di quanto il retrieval compositivo dipenda dall'esplicitazione delle relazioni topologiche.
- **Quota di Rumore nel Mining Semantico:** Il controllo incrociato con il giudice visivo CLIP (sezione 2.3.5) ha evidenziato che circa il 40% dei positivi proposti dal mining non presenta una forte somiglianza a livello di pixel. Questo rumore residuo riflette la discrepanza fondamentale tra *somiglianza di scenario* (es. due cucine diverse) e *somiglianza fotografica* (colori, illuminazione, angolazione).

- **Dipendenza dalla Qualità del Grafo in Ingresso:** L'efficacia della GCN presuppone uno scene graph ben formato. Quando il sistema viene alimentato da un estrattore generativo VLM (sezione 5.1), scene visive con elevato affollamento possono generare allucinazioni o descrizioni ad alto rumore lessicale, innescando una propagazione dell'errore nella fase di retrieval vettoriale.

6.1.4 Analisi dei pattern di fallimento (Failure Modes)

Dall'ispezione qualitativa dei casi di mancato retrieval a Top-1 emergono due pattern di errore ricorrenti:

1. **Saturazione da Nodi Generici:** Scene caratterizzate da un numero elevato di nodi di sfondo (es. *Sky*, *Wall*, *Floor*) tendono a polarizzare il global mean pooling, attenuando il peso relativo di oggetti rari ma altamente distintivi. L'introduzione di filtri di salienza a monte o di pooling pesato costituisce la contromisura naturale per questo fenomeno.
2. **Simmetria Topologica nelle Relazioni:** Nei grafi in cui due entità interagiscono con predicati simmetrici o complessi, la GCN può faticare a distinguere la direzionalità esatta se la connettività circostante è sparsa, portando al recupero di scene che condividono gli stessi oggetti in ruoli sintattici invertiti.

6.2 Conclusioni e sviluppi futuri

6.2.1 Sintesi del lavoro

Il progetto ha dimostrato l'efficacia dei grafi di scena arricchiti ontologicamente come rappresentazione strutturata e invariante per il *content-based image retrieval* semantico. L'architettura sviluppata ha integrato tecnologie del Semantic Web (Virtuoso) per espandere i nodi con tassonomie astratte, reti neurali a grafo (GINECONV e SAGECONV) per modellarne la topologia relazionale, e indici vettoriali FAISS ad alte prestazioni per la ricerca scalabile.

I risultati sperimentali evidenziano che l'approccio proposto supera nettamente le baseline visive non strutturate, staccando ResNet-50 (+36,7%–44,6%) e CLIP (+26,0%–37,2%) grazie alla capacità di catturare la composizionalità dei predicati d'azione. La combinazione vincente di convoluzione GINECONV, regolarizzazione ontologica e perdita Triplet Margin ha raggiunto un'accuratezza ad @20 dell'81,58% sul sotto-insieme controllato e del 67,20% sulla galleria estesa di 55k scene (con oltre 20.000 distrattori). Infine, le indagini avanzate hanno confermato una notevole robustezza a perturbazioni strutturali (87,20% ad @20 anche rimuovendo il 40% dei nodi), un'eccellente generalizzazione zero-shot (81,71%) e la piena interpretabilità geometrica dello spazio latente tramite GNNEXPLAINER.

6.2.2 Copertura degli obiettivi

La Tabella 11 mappa i requisiti progettuali minimi ed extra definiti nella traccia alle rispettive sezioni della relazione in cui sono stati discussi e soddisfatti.

Tabella 11: Mappatura degli obiettivi di progetto rispetto ai capitoli della relazione.

Tipologia	Obiettivo	Sezione
Minimo	Baseline visive (ResNet e CLIP)	3.1
Minimo	Graph Neural Network (GINE / SAGE)	3.2
Minimo	Loss contrastive (NT-Xent / Triplet)	3.3
Minimo	Motore di retrieval e metriche	4.2
Extra 1	Infrastruttura Cloud e Deployment	5.2
Extra 2	Robustezza del modello	4.4
Extra 3	Estrazione dinamica tramite VLM	5.1
Extra 4	Explainability (GNNEExplainer)	4.4

6.2.3 Sviluppi futuri

- **Perfezionamento del VLM e dell'estrazione:** Sostituzione o fine-tuning di modelli generativi avanzati per ridurre le *allucinazioni* spaziali nelle scene altamente affollate, stabilizzando ulteriormente la generazione dei JSON strutturati.
- **Cura dei dati e pre-processing:** Lavorare in modo più mirato sulla *scrematura* dei dati in ingresso, applicando filtri di salienza semantica ancor prima di darli in pasto all'encoder, in modo da scartare nodi irrilevanti o troppo generici.
- **Architetture avanzate:** Attenzione mirata sugli archi (Edge Attention) e pooling gerarchico al posto del semplice *mean pooling* globale.
- **Validazione umana:** Ground truth ottenuta da giudizi umani su un sottoinsieme, per validare matematicamente il criterio automatico di similarità semantica utilizzato per il training.
- **Valutazione end-to-end:** Misurazione quantitativa dell'intera catena (Immagine $\rightarrow$ VLM $\rightarrow$ grafo $\rightarrow$ retrieval) su benchmark standardizzati per quantificare accuratamente la propagazione dell'errore.

Riferimenti bibliografici

- [1] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Transactions on Big Data*, 2021.
- [2] Z. Ying *et al.*, "GNNEExplainer: Generating explanations for graph neural networks," in *NeurIPS*, 2019.

A Iperparametri completi

Tutti i valori numerici usati nelle esecuzioni definitive sono raccolti qui, così che il corpo della relazione possa citarli senza ripeterli. Tabella 12 elenca ciò che è comune a tutte le esecuzioni

e, nelle ultime righe, ciò che le distingue. I valori corrispondono ai default dei rispettivi script salvo dove lo script di job li sovrascrive esplicitamente.

Tabella 12: Iperparametri delle esecuzioni definitive. Le prime tre sezioni sono identiche in tutte le configurazioni, l’ultima è ciò che varia.

Gruppo	Parametro	Valore
Encoder	dimensione di ingresso	768 (embedding NOMIC dei nodi)
	dimensione nascosta	256
	dimensione di uscita	256, normalizzata ℓ_2
	numero di layer	3
	dropout	0,3
	pooling	media globale
	projection head	MLP $256 \rightarrow 256 \rightarrow 256$
Ottimizzazione	ottimizzatore	AdamW
	learning rate iniziale	10^{-3}
	weight decay	10^{-4}
	scheduler	cosine annealing, $T_{max} = \text{epoche}$
	epoche	300
	batch	128
Positivi	modalità	mixed (mining semantico)
	peso oggetti / triple	0,5 / 0,5
	soglia minima s	0,15
	positivi per anchor	al massimo 5
	veto semantico	0,55
	soglia di bypass	0,30
	oggetti minimi per anchor	3
	attenuazione dei generici	0,15
Augmentation	rimozione di nodi	10 %
	rimozione di archi	20 %
	mascheramento feature	10 %
Ciò che varia	convoluzione	SAGECONV oppure GINECONV
	loss	NT-Xent oppure Triplet Margin
	temperatura (NT-Xent)	0,2
	margin (Triplet)	0,3, con semi-hard mining

Due parametri meritano una precisazione, perché esistono nell’implementazione ma non sono attivi. Il primo è la soglia sul numero minimo di triple, lasciata a zero e discussa in sezione 2.3.2. Il secondo è la proiezione delle baseline a una dimensione comune, disponibile come argomento e non usata per le ragioni esposte in sezione 3.1.2.

B Riproducibilità

Questa appendice raccoglie i comandi effettivi e l’ambiente di esecuzione, in modo che la pipeline dal grafo al vettore possa essere ripercorsa senza ricostruirla dal codice.

B.1 Ambiente di esecuzione

Gli addestramenti sono stati eseguiti su un cluster con gestore di code SLURM, all'interno di un'immagine `apptainer` condivisa. Ogni job ha impegnato una quota di GPU **NVIDIA L40S**, quattro core e 16 GB di memoria, con un limite di dodici ore che nessuna esecuzione ha avvicinato, essendo tutte comprese tra i dieci e gli undici minuti. Le quattro esecuzioni sono inoltre finite sullo stesso nodo di calcolo, il che rende i tempi confrontabili tra loro. Le librerie principali sono Python 3.11, PyTorch 2.7.1 compilato per CUDA 11.8, `torchvision` 0.22.1, `transformers` 5.3.0 e `sentence-transformers` 5.2.3. `torch_geometric` è alla versione 2.8.0 e viene installato dal job a partire dal wheel incluso nel repository, dal momento che i nodi di calcolo non hanno accesso alla rete.

Il Mac di sviluppo è stato usato soltanto per la stesura del codice e per le verifiche su dataset ridotto, e la selezione automatica del dispositivo riconosce CUDA, Metal oppure CPU a seconda della macchina.

B.2 Semi e determinismo

La suddivisione del corpus è generata una volta sola con seed fisso pari a 42 e salvata su file, quindi tutte le esecuzioni leggono le stesse scene negli stessi insiemi. Anche la ground truth è deterministica per costruzione, essendo calcolata da regole su insiemi e conteggi. Non sono invece fissati il seed di inizializzazione dei pesi e quello che governa l'ordine dei batch, per cui una ripetizione dello stesso comando produce numeri leggermente diversi. È il limite già dichiarato in sezione 4.1.1, e la conseguenza pratica è che le differenze molto piccole tra configurazioni non vanno interpretate.

B.3 Comandi

```
1 sbatch script/DeepLearning/run_train_cluster.sh sage ntxent
2 sbatch script/DeepLearning/run_train_cluster.sh gine ntxent
3 sbatch script/DeepLearning/run_train_cluster.sh gine triplet
4 sbatch script/DeepLearning/run_train_cluster.sh sage triplet
```

Listing 1: Le quattro esecuzioni definitive. Lo script di job riceve architettura e loss come argomenti.

```
1 python script/DeepLearning/train_gcn.py \
2     --data Data/sceneGraph/embedded/train_scene_graphs.pt \
3     --conv gine --loss triplet --positives mixed \
4     --epochs 300 --batch-size 128 --lr 1e-3 \
5     --temperature 0.2 --min-sim 0.15 --pos-k 5 --sem-veto 0.55 \
6     --out checkpoints/gcn_encoder_gine_triplet.pth
```

Listing 2: Comando equivalente eseguito dentro al job, riportato per intero.

```
1 python script/DeepLearning/extract_gcn_embeddings.py \
2     --split test_gallery --ckpt checkpoints/gcn_encoder_gine.pth
3
4 python script/DeepLearning/extract_baseline_embeddings.py \
```

```

5  --encoder clip --split test_gallery
6
7  python script/DeepLearning/build_relevance.py \
8  --queries Data/sceneGraph/embedded/test_queries_scene_graphs.pt \
9  --gallery Data/sceneGraph/embedded/test_gallery_scene_graphs.pt \
10 --out Data/gcn/relevance_test.pt

```

Listing 3: Estrazione degli embedding e costruzione della ground truth di valutazione.

Ogni archivio prodotto conserva accanto ai vettori la lista degli identificatori di scena e, nel caso della ground truth, i parametri con cui è stata generata, così che un risultato possa essere ricondotto alla configurazione che lo ha prodotto senza doversi fidare della memoria.

C Esempi qualitativi di retrieval

Questa appendice raccoglie una selezione di query estratte durante la valutazione del modello migliore (`gine_triplet`), affiancando per ciascuna la relativa immagine sorgente, il grafo di scena interrogato e la *top-5* dei risultati restituiti dal motore FAISS. Questo confronto qualitativo permette di visualizzare concretamente cosa l'operatore convoluzionale e la logica contrastiva considerino "semanticamente simile".

C.1 Esempio di Retrieval 1: Successo su scena complessa

La Figura 14 mostra il caso in cui la query (a sinistra) ritrae una scena strutturata. Nonostante i risultati restituiti differiscano per stile visivo, il modello `gcn_gine_triplet` riesce a preservare fedelmente la struttura relazionale sottostante, recuperando cinque immagini dotate della stessa topologia semantica.

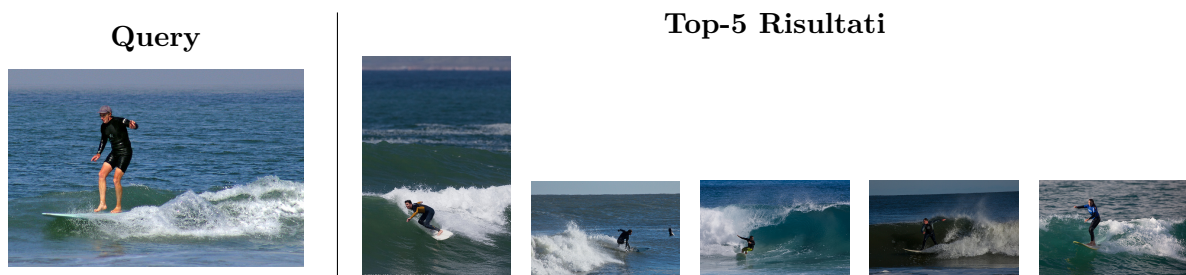


Figura 14: Successo su scena complessa: struttura relazionale mantenuta.

C.2 Esempio di Retrieval 2: Astrazione semantica

In questo secondo caso (Figura 15), viene testata la capacità del modello di astrarre il concetto visivo generale. I risultati recuperati includono variazioni dello stesso tema in contesti simili. La rete GNN dimostra di aver generalizzato i nodi principali verso categorie più ampie, catturando l'essenza semantica della scena piuttosto che la mera somiglianza a livello di pixel.



Figura 15: Astrazione semantica: generalizzazione degli oggetti urbani.

C.3 Esempio di Retrieval 3: Focus sui dettagli

La Figura 16 illustra una situazione in cui la ricerca premia la corrispondenza di dettagli fini e nodi periferici del grafo. Un elemento specifico presente nella query diviene il fattore discriminante della ricerca: i primi 5 risultati mostrano tutte interazioni simili, confermando il contributo vitale dell'arricchimento degli attributi svolto dall'estrattore VLM.

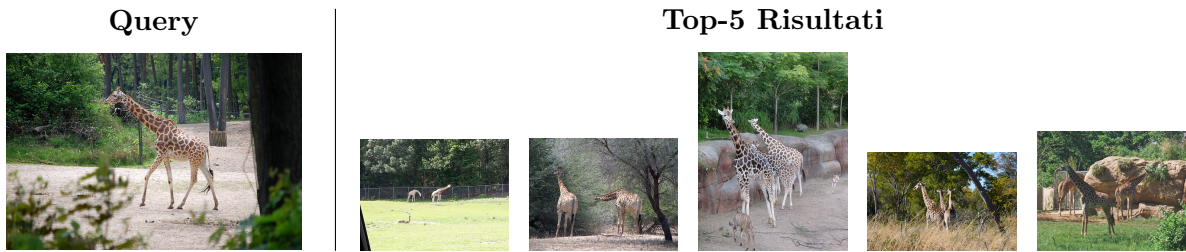


Figura 16: Focus sui dettagli: importanza delle micro-interazioni.

C.4 Analisi di un caso d'errore (Failure Case)

Nonostante le ottime prestazioni aggregate, l'approccio basato sui grafi è vulnerabile a specifici bias topologici.

Come osservabile nella Figura 17, il modello ha fallito visibilmente nel recuperare la scena corretta. L'errore è imputabile a un'"allucinazione" strutturale: l'eccessiva densità di nodi di sfondo (ad esempio pattern ripetuti o porzioni di scenario ampie) ha dominato il pooling dell'embedding. Di conseguenza, il motore FAISS ha restituito immagini che condividono lo stesso rumore di fondo ingarbugliato, ignorando del tutto l'azione del soggetto principale in primo piano.



Figura 17: Caso d'errore: il rumore di fondo domina l'embedding del grafo.